arXiv:2505.18369v2 [cs.LG] 2 Feb 2026

Small Models, Smarter Learning: The Power of Joint Task Training

Csaba Both [1 *] **Benjamin Hoover** [2 3] **Hendrik Strobelt** [3] **Dmitry Krotov** [3] **Daniel Karl I. Weidele** [3]
Mauro Martino [3] **Nima Dehmamy** [3 *]

Abstract

Multi-task learning improves generalization, but when does it reduce the model capacity required to learn? We provide a systematic study of how joint training affects the *learning transition*—the minimum model size at which a task can be learned—using nested arithmetic (ListOps) and permutation groups as controlled testbeds. Certain task pairings dramatically reduce model size requirements: combining easy operations (MAX, MIN, PROD) with hard ones (modular addition, permutation products) enables learning with 2–7× fewer parameters. Crucially, we also identify when synergies *fail*: pairing structurally similar hard tasks (e.g., ADD with alternating-sign NADD) provides no benefit, nor does pairing tasks lacking shared computational primitives. PCA of learned embeddings reveals that successful joint training induces structured number representations (ordering, parity, modular structure) absent in single-task models. Transfer experiments confirm these representations are causal: models pretrained on easy tasks learn addition at 7× smaller sizes. Our results establish that task compatibility—not mere diversity—determines whether joint training reduces capacity requirements, providing quantitative guidance for curriculum design.

1. Introduction

Scaling laws predict language model performance as a function of model size (Hoffmann et al., 2022), compute (Muennighoff et al., 2023), and dataset size (Hestness et al., 2017), but treat training data as homogeneous and do not explain how task composition shapes learning efficiency. Prior work has shown that multi-task learning improves generalization (Caruana, 1997; Ruder, 2017) and that joint arithmetic training can benefit individual tasks (Lee et al., 2024; Abedsoltan et al., 2025). However, a systematic understanding of *when* joint training helps versus fails—and by how much—remains lacking.

We address this gap by providing the first quantitative characterization of how task composition affects the *learning transition*: the minimum model size at which a task can be learned. Using nested arithmetic operations (ListOps) and permutation groups as controlled testbeds, we systematically vary task pairings and measure changes in transition points. Our key finding is that task compatibility, not mere task diversity, determines whether joint training reduces capacity requirements. Specifically, we identify task pairings that reduce model size requirements by 2–7×, as well as pairings that provide no benefit despite superficial similarity (Fig. 1).

Mathematical tasks provide an ideal testbed for this investigation: they allow precise control over task difficulty, quantitative evaluation of accuracy, and systematic combination of operations (Nangia & Bowman, 2018; Nye et al., 2021; Dziri et al., 2023). We use the ListOps dataset (Nangia & Bowman, 2018), which consists of nested operations such as `MAX(2,3,ADD(4,5))`, extended with modular arithmetic and permutation groups.

Summary of contributions. We study how small transformer models learn nested operations—maximum (MAX), minimum (MIN), median (MED), modular addition (ADD), modular product (PROD), and alternating addition-subtraction (NADD: $\sum_i (-1)^i x_i$)—both in isolation and jointly. We further extend to permutation groups, where products can be decomposed into sub-block operations. Our main findings are:

1. **Task difficulty hierarchy:** MAX/MIN are easiest; MED and PROD (non-prime modulus) are intermediate; ADD, NADD, and PROD (prime modulus) are hardest, requiring an order of magnitude more parameters.

2. **Synergies that work:** Pairing easy tasks (MAX, MIN, MED, PROD) with hard tasks (ADD) reduces

[*]Equal contribution [1]Northeastern University [2]Georgia Tech [3]IBM Research. Correspondence to: Csaba Both <both.c@northeastern.edu>, Nima Dehmamy <Nima.Dehmamy@ibm.com>.

Preprint. February 3, 2026.

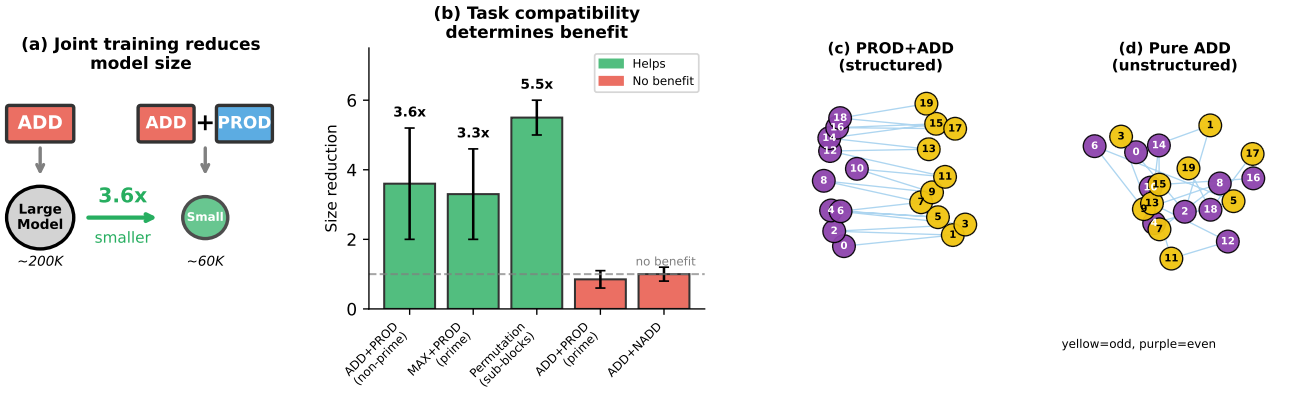


Figure 1. **Summary of main findings.** (a) Joint training on ADD+PROD reduces minimum model size by 3.6× compared to ADD alone. (b) Task compatibility determines benefit: easy+hard pairings help (green), while hard+hard pairings provide no benefit (red). (c,d) PCA of learned embeddings: joint training (PROD+ADD) induces structured representations with clear parity separation, while single-task training (pure ADD) yields unstructured embeddings.

model size requirements by 2–7×. Pretraining on MAX+MED then switching to ADD achieves similar reductions.

3. **Synergies that fail:** Pairing structurally similar hard tasks (ADD+NADD) provides no benefit. Nor does pairing ADD with PROD when the modulus is prime. This indicates that task synergy depends on shared computational primitives, not superficial similarity.

4. **Mechanistic evidence:** PCA of learned embeddings shows that successful joint training induces structured number representations (ordering, parity, divisibility) absent in single-task models.

5. **Generalization beyond arithmetic:** The same effect appears in permutation groups: joint training with sub-block operations reduces model size by 5–6×.

These results demonstrate that joint training can guide models toward more efficient solutions, revealing structure in scaling laws that depends critically on task composition. Understanding these synergies has practical implications: if certain task combinations make models "smarter" by inducing better representations, this could inform curriculum design for pretraining—a form of pre-pretraining that reduces the resources needed for downstream tasks.

2. Related Work

Multi-task learning. Multi-task learning improves generalization by sharing representations across tasks (Caruana, 1997; Ruder, 2017). Recent work has shown benefits for arithmetic: Lee et al. (2024) demonstrated that joint training on multiple arithmetic operations improves each task through shared number representations, and Abedsoltan et al. (2025) showed that compositional joint training leads to sudden performance jumps. However, these studies focused on task combinations that help, without systematically testing which pairings *fail*. Our work fills this gap: we identify specific task pairings that help (easy+hard) versus those that fail (hard+hard), and quantify the effect on minimum model size—showing reductions of 2–7×, not just accuracy improvements.

Curriculum learning. Presenting tasks in order of difficulty improves learning (Bengio et al., 2009; Yin et al., 2024; Kim & Lee, 2024). Our transfer experiments (Section 4.3) relate to curriculum learning, showing that pretraining on easy tasks enables learning hard tasks at smaller model sizes. However, our focus is on simultaneous joint training rather than sequential curricula.

Grokking and mechanistic interpretability. Work on grokking (Power et al., 2022; Liu et al., 2022) and mechanistic interpretability (Nanda et al., 2023) has shown that models solving modular arithmetic develop structured internal representations. We build on these insights by using embedding analysis to understand *why* joint training helps: successful task pairings induce structured embeddings (ordering, parity) that single-task training does not. Our setup differs from grokking studies in that we vary model size at fixed dataset size, rather than observing delayed generalization at fixed model size.

We provide the first systematic map of task compatibility for arithmetic operations, extending to permutation groups and confirming robustness to architectural choices (Appendix H).

3. Methodology

We train small-scale transformer models and analyze both their performance and internal representations. This sec-

tion outlines our methodology, detailing the dataset, model architecture, and evaluation protocols.

Choice of task. ListOps consists of nested mathematical equations involving operations. Our setup uses MAX, MIN, MED (median), ADD (addition modulo n), PROD (product modulo n) and NADD (alternating addition and subtraction modulo n) applied to numbers (0 to $n-1$). We conduct experiments using $n = 10$ to $n = 226$ (Appendix F.3.1). ListOps provides procedural generation with controlled difficulty, exact evaluation, adjustable complexity via nesting depth, and systematic investigation of inter-task relationships.

Dataset description. We use a simplified functional notation of the form $f(x, y, \ldots)$. For example, `max_10(3,min_10(7,4,9))=4<eos>`, where `<eos>` denotes the end token. The subscript in the function name indicates the modulo operation e.g., `max_10` represents the `max` function modulo 10.

Tokenization and CoT. We use word-level tokenization: each operation (e.g., `add_20`), each number (e.g., `13`), and each symbol (`(`, `)`, `=`, `<eos>`) is a single token (Appendix F.3.2). Since directly solving nested expressions proves challenging (Fig. 24), we use chain-of-thought (CoT) that decomposes expressions into intermediate steps (Appendix F.3.3): `add_10(9 add_10(3 4))>add_10(9 7)>6=6<eos>` where > indicates solving the innermost operation. This follows the scratchpad approach of Lee et al. (2024). Task difficulty increases with the number of operands and nesting depth (Appendix Fig. 48).

Train-Test split. The experiments were conducted on data generated with a maximum nesting depth of 2 and up to 3 operands per operation. Equations are constructed from digit sequences forming duplets and triplets (e.g., "1,0", "0,0,1"). Since the CoT format can produce many intermediate patterns within a single equation, we implement a rigorous train-test split to prevent data leakage: we designate 100 of the 1000 possible triplets (e.g. "749") as an exclusion set. Training equations never contain any excluded triplet, while test equations each contain at least one excluded triplet, ensuring that test set patterns are novel.

Permutation groups. To test generality beyond arithmetic, we extend ListOps to permutation groups. By Cayley's theorem, every finite group is isomorphic to a permutation subgroup, making this a natural generalization. We construct 6×6 block-diagonal permutation matrices (two 3×3 blocks) and define three operations: `OP` (full matrix, 36 elements), `OP_TOP` and `OP_BOTTOM` (individual blocks, 6 elements each). See Fig. 9a and Appendix Figs. 15–13 for details.

Performance evaluation. We randomly select 1000 equations from the held-out test set. The model is prompted to generate solutions character-by-character, starting from the equation prompt (e.g., `add_10(3,min_10(7,4,9))>`). We use the output to produce two metrics: **Loss:** using cross-entropy computed for every character of the output. **Accuracy:** based on the number of correct answers evaluated using *only the final answer*, which we define as the first character after the first '=' symbol.

Model architecture. For our experiments, we employed a series of tiny GPT models, inspired by the nanoGPT architecture (Karpathy, 2022). Unlike the standard sequential transformer, we implemented a recurrent variant in which a single transformer block was iteratively applied by feeding back its output as the next input (see the architecture Appendix Fig. 23). This recurrent design improved learning efficiency and yielded more structured embedding patterns (see results from traditional deep transformer models in the Appendix F). Each model in our study uses a single attention head. We set the feedforward hidden dimension to four times the embedding dimension, a common practice in transformer architectures, providing sufficient complexity in the feedforward networks while keeping the model size constrained. By varying the embedding size we create a range of models with different parameter counts. Ablation studies with 1–4 attention heads show no qualitative change in results (Appendix H), and experiments with standard deep transformers (2–4 layers) yield similar patterns, though with less structured embeddings (Appendix F).

Learning transition. We define the *learning transition* as the minimum model size at which a task can be learned. Operationally, we fit a logistic function to accuracy vs. model size and define the transition point as the model size reaching 80% of the asymptotic accuracy. We chose 80% rather than 50% because the latter corresponds to models that have not yet reliably mastered the task; 80% ensures the model has learned a generalizable solution rather than partially memorizing patterns. The transition curves are remarkably consistent across runs for the recurrent architecture with fixed dataset size.

Dataset scaling. When varying the modulo n, we scale the dataset size linearly with n. This choice is motivated by the observation that smaller moduli (e.g., $n = 10$) can be learned with relatively little data, while larger moduli (e.g., $n = 113$) require substantially more examples to avoid underfitting. We found that linear scaling produces transition points that grow slowly (often sub-linearly) with

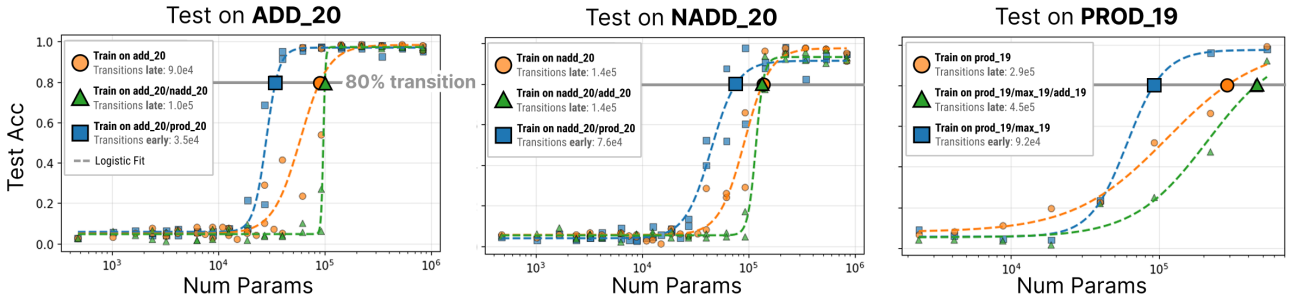


Figure 2. **Joint training with compatible tasks reduces the model size needed to learn.** Each panel shows accuracy vs. model size for operations trained in isolation (●), with an easier task (■), or with a harder task (▲). **Left:** ADD benefits from PROD (2.5× smaller models) but not from NADD. **Center:** NADD benefits from PROD but not from ADD. **Right:** PROD benefits from MAX (2.6× smaller) but not from ADD+MAX. Dashed lines show logistic fits; legends report the 80% accuracy transition point.

n for most task combinations (Fig. 8), suggesting that data complexity requirements increase more slowly than linearly. Importantly, the *relative* benefits of joint training—the 2–7× reduction in model size—are consistent across moduli, indicating that our findings are robust to this design choice.

4. Results

To understand how language models acquire mathematical abilities, we focus on accuracy as the primary metric for observing the emergence of learning, following the approach of the emergent abilities literature (Wei et al., 2022). While some subsequent studies have questioned the concept of emergence by examining other metrics (Schaeffer et al., 2023), we argue that this critique overlooks similar patterns observed in physics: in a phase transition, only some quantities may exhibit discontinuous changes. A metric that can capture the emergence of the new phase is called the "order parameter". Hence, we choose to use accuracy as our order parameter.

Experiments use a recurrent transformer with embedding dimension n_{embed} ranging from 8 to 362, and 4 recurrence steps (we have found that using 4 recurrence steps yields the best performance). Similar to the emergence literature, we observe the total number of parameters to be the strongest indicator of accuracy, rather than n_{embed} or depth. Our experimental design focuses on identifying the minimum model size sufficient for learning each task or task combination—once this threshold is exceeded, larger models reliably reach high accuracy, making further scaling unnecessary for our analysis. We primarily present results for modulo 20, with similar patterns observed for other modulos (Appendix F.5 - mod 10 results, F.4 - mod 26 results). Experiments with higher moduli reveal a basic scaling relationship: larger moduli require proportionally larger models and more training data to achieve comparable performance.

We find that joint training on compositional tasks substantially improves performance and reduces the parameter requirements for learning individual operations. Training on multiple arithmetic tasks helps models learn the basic building blocks needed for solving more complex problems. This shared representation not only accelerates learning but also produces abrupt transitions in performance once the necessary constituent skills are mastered (Lubana et al., 2024; Okawa et al., 2023). However, joint training is not universally beneficial, and task compatibility is more nuanced than surface-level similarity suggests. Our experiments reveal that effective transfer requires overlap in underlying computational primitives and mathematical structures, not merely similar algorithms or comparable difficulty. Pairing hard problems together often increases required model size when tasks lack shared low-level building blocks, demonstrating that joint training can actively interfere with learning rather than facilitate it. These findings extend previous work on compositional generalization (Lee et al., 2024) by systematically characterizing when multi-task learning helps or hinders algorithmic task performance, showing that compatibility depends on deep structural properties rather than intuitive task similarity.

Transition to learning. Figure 2 shows learning transitions for single and joint operations. We find that joint training enables models to solve more difficult tasks with fewer parameters and less data. For example, ADD and NADD are the most demanding tasks when trained in isolation, yet when combined with PROD, models that are ×2.5 smaller are able to reach 100% accuracy on the task. Similarly, while PROD on prime moduli is challenging on its own, pairing it with MAX reduces the required model size by a factor of ×2.6. In contrast, pairing ADD with NADD does not decrease the required model size, nor does pairing PROD-prime with ADD or NADD.

These results challenge the assumption that harder tasks always demand larger models. Joint training helps only when tasks share the right structure: ADD and NADD use similar algorithms but require different representations, so pairing

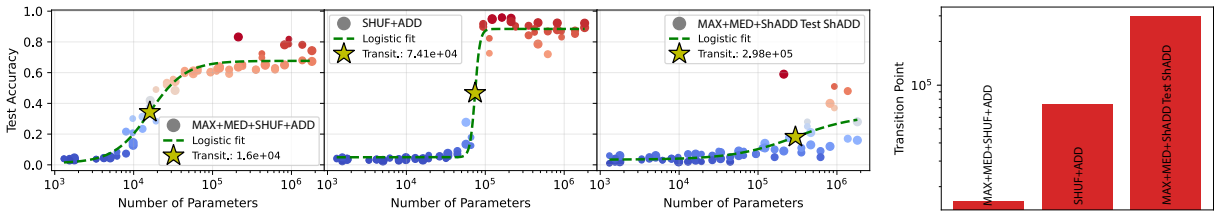


Figure 3. **Shuffled ADD: joint training fails when number structure is destroyed. Left:** Learning curves for various tasks including Shuffled ADD (randomly permuted addition table). **Right:** Transition points—Shuffled ADD is nearly as hard as pure ADD. Unlike regular ADD, joint training with MAX+MED *hurts* Shuffled ADD: the mixed model reaches only ∼20% accuracy (third panel). This confirms that MAX+MED induces number-like embeddings incompatible with the randomized table.

them fails. Meanwhile, PROD with non-prime modulus helps ADD because divisibility patterns create shared computational primitives. The key insight: task synergy requires overlapping low-level building blocks, not superficial similarity. These patterns extend to combinations of 3–4 tasks: joint training on all combinations of MAX, MIN, MED, and ADD (modulo 26) shows the same grouping of transition points by task compatibility (Appendix Fig. 29).

Testing the number properties hypothesis. We hypothesize that joint training helps because easier tasks reveal number properties (e.g., ordering, parity) that benefit harder tasks. To test this, we designed an experiment that destroys number relationships: we shuffle the addition table so that $A + B = C$ becomes a random mapping with no arithmetic structure (Appendix D). If joint training benefits ADD by revealing number properties, then Shuffled ADD should not benefit from joint training with MAX and MED.

We conducted experiments using a shuffled, symmetric ADD table ($A + B = B + A$) in modulo 26. The shuffled addition proved more difficult to learn than all other mathematical operations except the original ADD (Fig. 3). Crucially, MAX+MED+Shuffled ADD was *harder* than Shuffled ADD alone: the mixed model reached only 60% accuracy on Shuffled ADD and required twice as many parameters. This is the opposite of what we observe with regular ADD, where joint training helps dramatically.

This confirms our hypothesis: MAX and MED induce number-like embeddings with ordering structure, but Shuffled ADD is incompatible with these representations. PCA analysis shows Shuffled ADD embeddings lack parity separation or ordering (Appendix Fig. 22).

4.1. Embedding Analysis: What Number Properties Emerge?

The Shuffled ADD experiment confirms that number properties are key to joint training benefits. We now examine *what specific properties* emerge in the learned embeddings. To do this, we analyze the embedding layer using principal component analysis (PCA).

PCA methodology. In our tokenization scheme, each number and operation is a single token. While embedding layers of different-sized models cannot be directly compared (they have different dimensions), we can compare their $n_{\text{vocab}} \times n_{\text{vocab}}$ cosine similarity matrices. To reduce noise, we average similarity matrices across models that learned each task (accuracy $> 90\%$), then compute PCA on the averaged matrix. This reveals consistent patterns across model sizes (see Appendices F.4 and F.5 for additional results).

Structured embeddings in joint training. Figure 4 shows that joint training induces structured embeddings: PROD shows strong parity separation and ordering; ADD+PROD preserves this structure; pure ADD shows weak structure; NADD develops different structure; and ADD+NADD loses structure from both tasks. This explains why ADD+NADD fails despite algorithmic similarity: incompatible representations.

Quantifying embedding structure: separation score. Beyond parity, we observe that embeddings cluster numbers by their remainders modulo divisors of n. To quantify this, we define a *separation score*: for each candidate divisor d, we group numbers by their remainder mod d, compute centroids for each group in the top PCs, and measure the ratio of between-group distance to within-group standard deviation. High separation scores indicate strong clustering by that divisor (Fig. 5).

The results reveal: PROD embeddings cluster by divisors of n (e.g., mod-2, 3, 6 for n=42); prime moduli show no clustering, explaining why PROD-prime is hard; and ADD+PROD inherits PROD's divisor structure while pure ADD shows weak clustering. This explains why PROD helps ADD only for non-prime moduli: divisibility structure creates transferable primitives.

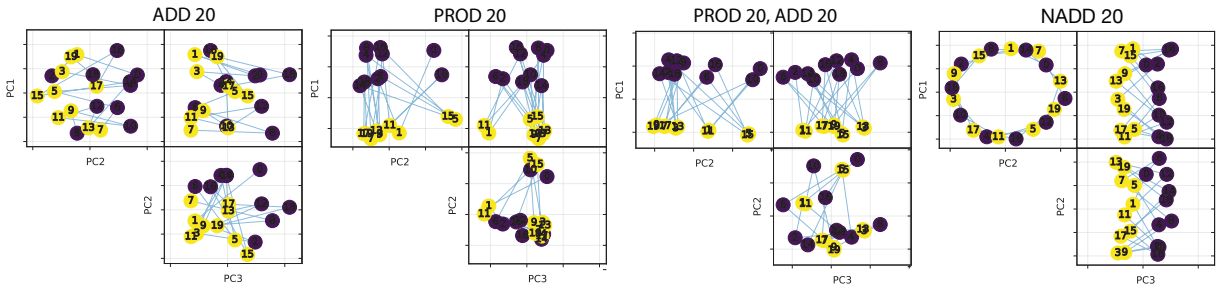


Figure 4. **PCA of embeddings reveals structured number representations in joint training.** Top 5 principal components of the cosine similarity matrix, colored by parity (yellow=odd, purple=even). **PROD:** Strong parity separation and ordering structure. **ADD+PROD:** Preserves PROD's structure, explaining why joint training helps ADD. **Pure ADD:** Weak, noisy structure. **NADD:** Highly structured but different from ADD. **ADD+NADD:** Loses structure from both tasks, explaining why this pairing fails.

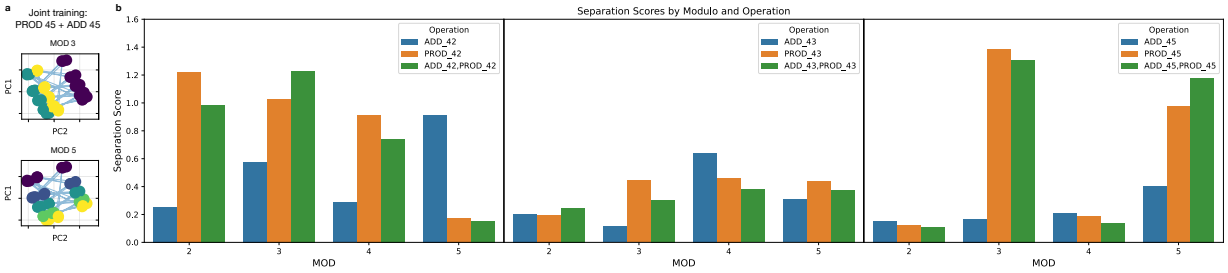


Figure 5. **Embeddings cluster by divisors of the modulus. (a)** PCA of PROD-45+ADD-45 embeddings colored by remainder class shows clear mod-3 and mod-5 clustering (divisors of 45). **(b)** Separation score (between-centroid distance / within-group variance) quantifies clustering strength across moduli. Non-prime moduli (42, 45) show strong clustering by their divisors; prime modulus (43) shows none. ADD+PROD inherits PROD's divisibility structure; pure ADD shows weak clustering. This explains why PROD helps ADD only for non-prime moduli.

4.2. Learning Dynamics: How Structure Emerges

The embedding analysis shows that joint training produces structured representations. But *when* does this structure emerge during training? We examine the learning dynamics to understand how easy tasks scaffold hard tasks.

Figure 6 compares pure ADD vs. MAX+MED+ADD ($n_{\text{embed}} = 96$). In joint training, MAX learns first, then MED, then ADD—and structured embeddings emerge *before* ADD accuracy rises, suggesting structure enables learning. Joint training converges in half the steps; pure ADD shows no structure and loss plateaus.

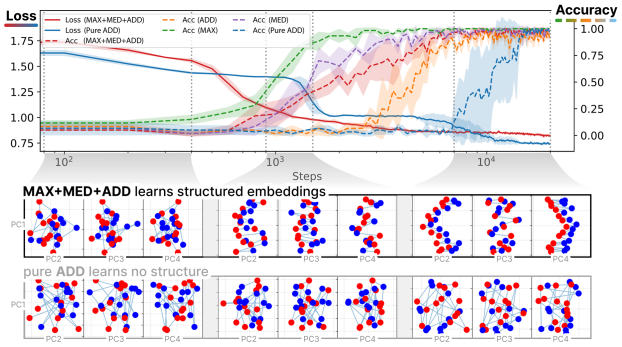


Figure 6. **Joint training improves training speed by encouraging structured embeddings during learning dynamics.** (Top) Two models are trained: one on a joint MAX+MED+ADD dataset and one on a pure ADD dataset (mod 26). We observe that both training loss (solid) and accuracies (dashed) improve *faster* for the joint training model across all evaluated tasks (mean $\pm\sigma$ over 3 runs.). In joint training, MAX learns first, then MED, then ADD learns only after the embeddings become structured. (Bottom) PCA snapshots are taken during training (number parity is colored red/blue). Joint training accuracies rise quickly after the embeddings learn structure, which only happens in the joint training model. The pure ADD model shows no embedding structure and exhibits loss plateaus, suggesting it struggles to discover an efficient algorithm.

4.3. Causal Test: Can Learned Structure Transfer?

To establish causation, we test whether structure from easy tasks *transfers* to hard tasks. We design a curriculum (Fig. 7): train on ADD+PROD, gradually shift to pure ADD, and compare to a baseline trained only on ADD.

The results are striking: baseline ADD stays at $\sim$15% accuracy, but curriculum achieves $>$95%. Structured embeddings develop during ADD+PROD and persist through the transition.

We also verified this effect with other task combinations:

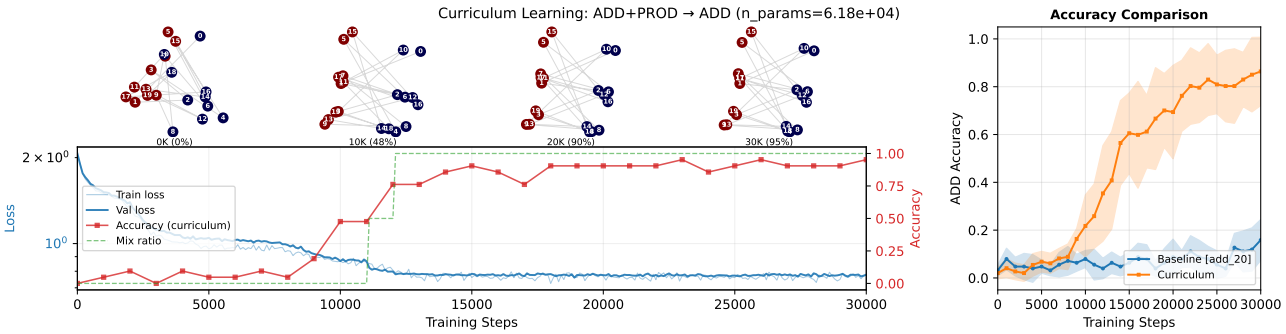


Figure 7. **Curriculum learning enables transfer of structured representations. Left:** PCA of embeddings at 0K, 10K, 20K, and 30K steps (top), colored by parity (red=odd, blue=even). Structure emerges during joint ADD+PROD training (mix ratio rises at 10K) and persists as the curriculum shifts to pure ADD. Loss and accuracy curves (bottom) show the curriculum transition. **Right:** Baseline ADD stays at ∼15% accuracy; curriculum reaches >95%. Shaded: $\pm 1\sigma$ over runs.

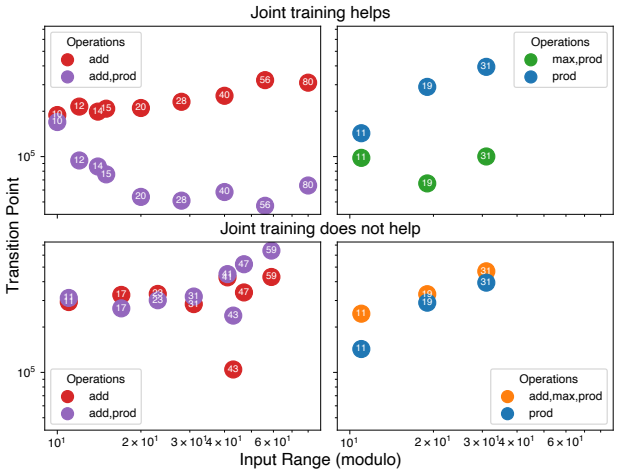


Figure 8. **Joint training helps only with compatible tasks** when scaling across moduli. **Top (helps):** ADD+PROD (left) reduces transition points by $3.6 \pm 1.6\times$ for non-prime moduli, while MAX+PROD (right) reduces PROD transition by $3.3 \pm 1.3\times$ even for prime moduli. **Bottom (no benefit):** ADD+PROD (left) provides no benefit for prime moduli (ratio $0.85 \pm 0.25\times$), while ADD+MAX+PROD (right) *increases* PROD-prime transition (ratio $0.77 \pm 0.13\times$). Modulus values are indicated on the x-axes and as numbers on each scattered point.

pretraining on MAX+MED followed by transition to pure ADD yields similar results, with models $7\times$ smaller than the pure ADD transition successfully learning the task (see Appendix J for details).

4.4. Scaling Across Moduli

Do the benefits (and failures) of joint training persist across different moduli? We systematically measure transition points for various task combinations across moduli ranging from 10 to 80 (Fig. 8). The results cleanly separate into cases where joint training helps versus cases where it does not.

Joint training helps: ADD+PROD reduces transitions by $3.6 \pm 1.6\times$ for non-prime moduli; MAX+PROD reduces PROD transitions by $3.3 \pm 1.3\times$ even for primes (MAX provides ordering structure).

Joint training fails: ADD+PROD provides no benefit for prime moduli ($0.85 \pm 0.25\times$); ADD+MAX+PROD actually *hurts* PROD-prime ($0.77 \pm 0.13\times$). These patterns hold robustly across moduli (Appendix Fig. 44).

4.5. Generalization Beyond Arithmetic: Permutation Groups

To test generality, we extend ListOps to permutation groups with no numerical structure. We construct 6×6 block-diagonal permutation matrices (two 3×3 blocks) with three operations: `OP` (full matrix, 36 elements), `OP_TOP` and `OP_BOTTOM` (individual blocks, 6 elements). The sub-block operations decompose the full operation into simpler components—analogous to how MAX helps ADD.

Joint training yields a **5–6× reduction** in model size (Fig. 9b,c). By Cayley's theorem, every finite group is isomorphic to a permutation subgroup, suggesting these principles apply broadly: decomposing complex operations into simpler primitives helps learning, regardless of numerical structure.

5. Discussion

Why does joint training help? Our results suggest that joint training benefits coincide with better representation learning: easier tasks like PROD and MAX restrict the embedding space to representations capturing number properties, which then support harder tasks like ADD. This aligns with grokking literature (Power et al., 2022; Liu et al., 2022), where successful generalization coincides with organized internal representations. The path to learning matters: mod-

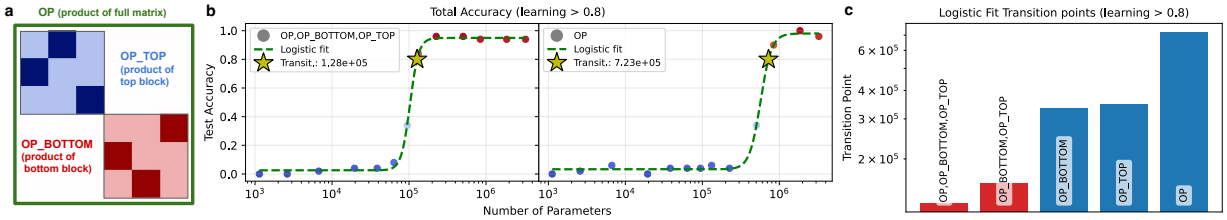


Figure 9. **Joint training benefits generalize to permutation groups. (a)** Setup: 6×6 block-diagonal permutation matrices with two 3×3 blocks. OP acts on the full matrix (36 elements); OP_TOP and OP_BOTTOM act on individual blocks (6 elements each). **(b)** Transition curves: joint training vs. pure OP. **(c)** Sub-block operations reduce the OP transition point by 5–6×, confirming that decomposing complex operations into simpler primitives helps learning—even in non-numerical domains.

els developing structured embeddings early require fewer resources. Our transfer experiments show models pretrained on MAX+MED learn ADD at $7\times$ smaller sizes.

Toward a predictive theory: output distributions. We hypothesize that task difficulty relates to the output distribution of $\text{Op}(a, b) = c$: tasks with uniform output distributions (like ADD) offer no patterns to exploit, while tasks with peaked distributions (like PROD at non-prime moduli) allow divide-and-conquer strategies—learning frequent outputs first, then rarer ones. This explains why PROD difficulty changes drastically with modulus: at primes, the output distribution is nearly uniform; at non-primes, divisibility creates structure. The parity and modular clustering we observe in embeddings may emerge from such divide-and-conquer: e.g., in ADD_{20}, even+even and odd+odd both yield even results, creating exploitable structure. Our separation score (Fig. 5) provides quantitative evidence: tasks inducing higher separation by divisors tend to help harder tasks more.

Beyond arithmetic. Our permutation group results suggest these effects extend beyond arithmetic. By Cayley's theorem, every finite group is isomorphic to a permutation subgroup, so these principles may apply to a large class of algebraic tasks.

Connections to curriculum learning. Joint training and curriculum learning both exploit task relationships. Our transfer experiments show that sequential curricula benefit from the same mechanism: structured representations learned on easy tasks transfer to hard tasks. This suggests a general principle: pair tasks sharing computational primitives, not merely similar difficulty.

Algorithmic complexity. Minimum model size depends not just on task complexity but on the *algorithmic complexity* of the discovered solution—brute-force memorization requires far more resources than efficient computation. Joint training appears to bias models toward efficient algorithms by constraining the representation space.

Broader applicability. Prior work shows training arithmetic with text improves accuracy (Lee et al., 2024), and structured patterns help downstream tasks (Yin & Yin, 2024; Zhang et al., 2024), suggesting these principles may transfer across domains.

Limitations. Our synthetic tasks may not directly transfer to natural language with ambiguous outputs, though they suggest principles for curriculum design in pretraining. We focus on small models; effects at larger scales remain untested. Our PCA-based analysis captures only linear structure, and while our output distribution hypothesis provides partial predictions, a complete theory of task compatibility remains future work.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. Our findings suggest that strategic task composition can reduce model size requirements by 2–7×, potentially enabling more energy-efficient training and reducing computational costs. We study synthetic tasks and do not foresee direct negative societal consequences from this basic research.

References

Abedsoltan, A., Zhang, H., Wen, K., Lin, H., Zhang, J., and Belkin, M. Task generalization with autoregressive compositional structure: Can learning from d tasks generalize to d^T tasks? *arXiv preprint arXiv:2502.08991*, 2025.

Allen-Zhu, Z. and Li, Y. Physics of language models: Part 3.3, knowledge capacity scaling laws. *arXiv preprint arXiv:2404.05405*, 2024.

Bengio, Y., Louradour, J., Collobert, R., and Weston, J. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pp. 41–48, 2009.

Caruana, R. Multitask learning. *Machine learning*, 28(1): 41–75, 1997.

Dave, N., Kifer, D., Giles, C. L., and Mali, A. Investigating symbolic capabilities of large language models. *arXiv preprint arXiv:2405.13209*, 2024.

Dziri, N., Lu, X., Sclar, M., Li, X. L., Jiang, L., Lin, B. Y., Welleck, S., West, P., Bhagavatula, C., Le Bras, R., et al. Faith and fate: Limits of transformers on compositionality. *Advances in Neural Information Processing Systems*, 36, 2023.

Frieder, S., Pinchetti, L., Griffiths, R.-R., Salvatori, T., Lukasiewicz, T., Petersen, P., and Berner, J. Mathematical capabilities of chatgpt. *Advances in neural information processing systems*, 36, 2023.

Hestness, J., Narang, S., Ardalani, N., Diamos, G., Jun, H., Kianinejad, H., Patwary, M. M. A., Yang, Y., and Zhou, Y. Deep learning scaling is predictable, empirically. *arXiv preprint arXiv:1712.00409*, 2017.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. d. L., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

Karpathy, A. nanogpt. https://github.com/karpathy/nanoGPT, 2022. Accessed: 2024-09-24.

Kim, J. and Lee, J. Strategic data ordering: Enhancing large language model performance through curriculum learning. *arXiv preprint arXiv:2405.07490*, 2024.

Lee, N., Sreenivasan, K., Lee, J. D., Lee, K., and Papailiopoulos, D. Teaching arithmetic to small transformers. In *The Twelfth International Conference on Learning Representations*, 2024.

Liu, Z., Kitouni, O., Nolte, N. S., Michaud, E., Tegmark, M., and Williams, M. Towards understanding grokking: An effective theory of representation learning. *Advances in Neural Information Processing Systems*, 35:34651–34663, 2022.

Lubana, E. S., Kawaguchi, K., Dick, R. P., and Tanaka, H. A percolation model of emergence: Analyzing transformers trained on a formal language. *arXiv preprint arXiv:2408.12578*, 2024.

Muennighoff, N., Rush, A., Barak, B., Le Scao, T., Tazi, N., Piktus, A., Pyysalo, S., Wolf, T., and Raffel, C. A. Scaling data-constrained language models. *Advances in Neural Information Processing Systems*, 36, 2023.

Nanda, N., Chan, L., Lieberum, T., Smith, J., and Steinhardt, J. Progress measures for grokking via mechanistic interpretability. *arXiv preprint arXiv:2301.05217*, 2023.

Nangia, N. and Bowman, S. R. Listops: A diagnostic dataset for latent tree learning. *arXiv preprint arXiv:1804.06028*, 2018.

Nye, M., Andreassen, A. J., Gur-Ari, G., Michalewski, H., Austin, J., Bieber, D., Dohan, D., Lewkowycz, A., Bosma, M., Luan, D., et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.

Okawa, M., Lubana, E. S., Dick, R., and Tanaka, H. Compositional abilities emerge multiplicatively: Exploring diffusion models on a synthetic task. *Advances in Neural Information Processing Systems*, 36:50173–50195, 2023.

Petruzzellis, F., Testolin, A., and Sperduti, A. Benchmarking gpt-4 on algorithmic problems: A systematic evaluation of prompting strategies. *arXiv preprint arXiv:2402.17396*, 2024.

Power, A., Burda, Y., Edwards, H., Babuschkin, I., and Misra, V. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

Ruder, S. An overview of multi-task learning in deep neural networks. *arXiv preprint arXiv:1706.05098*, 2017.

Schaeffer, R., Miranda, B., and Koyejo, S. Are emergent abilities of large language models a mirage? *Advances in Neural Information Processing Systems*, 36, 2023.

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.

Yin, D. S. and Yin, X. Scaffolding learning: From specific to generic with large language models. *PloS one*, 19(9): e0310409, 2024.

Yin, S., You, W., Ji, Z., Zhong, G., and Bai, J. Mumathcode: Combining tool-use large language models with multi-perspective data augmentation for mathematical reasoning. *arXiv preprint arXiv:2405.07551*, 2024.

Zhang, S., Patel, A., Rizvi, S. A., Liu, N., He, S., Karbasi, A., Zappala, E., and van Dijk, D. Intelligence at the edge of chaos. *arXiv preprint arXiv:2410.02536*, 2024.

Zhong, Z., Liu, Z., Tegmark, M., and Andreas, J. The clock and the pizza: Two stories in mechanistic explanation of neural networks. *Advances in neural information processing systems*, 36:27223–27250, 2023.

A. Related works

Scaling laws for language models characterize how validation loss scales with model size (number of parameters) (Hoffmann et al., 2022), compute (FLOPs) (Muennighoff et al., 2023), dataset size (number of examples) (Hestness et al., 2017), and information capacity (in bits) (Allen-Zhu & Li, 2024), offering a foundation for model design. However, this perspective overlooks critical aspects of learning. Our results show that the composition of the training data—specifically, the mix of tasks—can significantly alter what and how a model learns. Even models with identical sizes can follow different learning trajectories depending on the training setup, suggesting that task structure shapes the internal algorithms that emerge during training.

Joint training Joint training on compositional tasks allows models to acquire the foundational primitives necessary for solving complex operations, often leading to abrupt performance improvements once all constituent skills are learned (Lubana et al., 2024; Okawa et al., 2023). Similarly, training on multiple arithmetic tasks has been shown to improve accuracy on individual operations, highlighting the benefits of shared representations for compositional generalization (Lee et al., 2024).

Number representation Neural networks trained on modular addition tasks develop structured embedding spaces that reflect underlying arithmetic operations. Mechanistic analyses have shown that small models can exhibit ordered number patterns and implement distinct algorithmic strategies, depending on initialization and hyperparameters (Zhong et al., 2023). Some models converge to known solutions such as the Clock algorithm, while others discover novel procedures like the Pizza algorithm, illustrating the algorithmic diversity that can emerge from fixed training data. Periodic structures in the embeddings can be characterized via Fourier analysis, offering additional interpretability (Nanda et al., 2023). These behaviors have also been linked to grokking dynamics, where models abruptly generalize after extended training, accompanied by the emergence of structured embedding patterns (Power et al., 2022; Liu et al., 2022).

In general language models performs poorly on symbolic mathematical tasks (Frieder et al., 2023; Dziri et al., 2023; Dave et al., 2024) such as the ListOps dataset (Nangia & Bowman, 2018) used in this study. Models often struggle with generalization, tending to memorize tasks rather than simulate the underlying algorithms. While mathematical tasks prove challenging for large language models to learn, they provide a controllable playground to test how models learn different tasks and to evaluate their accuracy quantitatively. Furthermore it allows us to tune the task difficulty by combining different operations.

Here, we investigate how language models acquire arithmetic skills by training small models on the ListOps dataset, which enables explicit evaluation through structured mathematical expressions. Adopting a bottom-up approach, we find that models learn to solve these tasks once the number of trainable parameters surpasses a critical threshold. This threshold shifts with the difficulty of the target operation (MAX = MIN $<$ MED $<$ ADD), indicating a dependency on task difficulty. Surprisingly, joint training on multiple operations often facilitates learning, outperforming models trained on individual operations (e.g. MAX+MED+SUM $<$ ADD). This suggests that task diversity can ease optimization by promoting shared representations.

Supplemental Material

B. Additional plots

B.1. Modulo 20

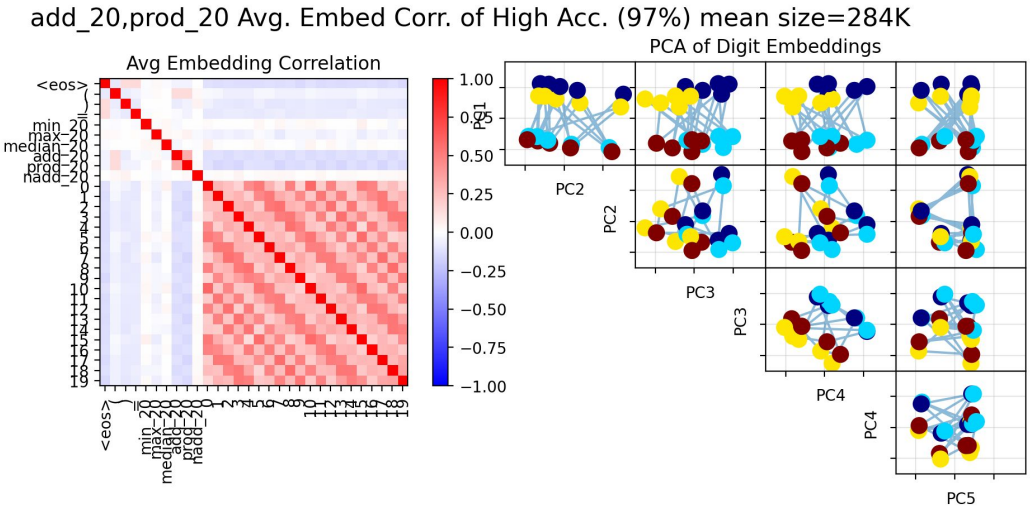


Figure 10. **ADD+PROD embeddings and PCs. PCs colored based modulo 4.**

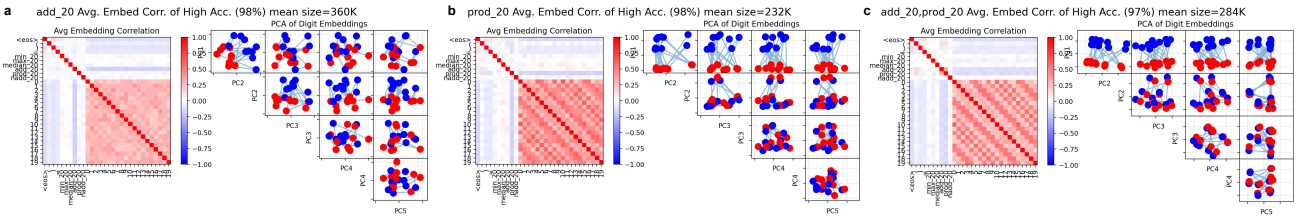


Figure 11. **ADD, PROD, ADD+PROD embeddings and PCs.**

B.2. Modulo prime - PROD

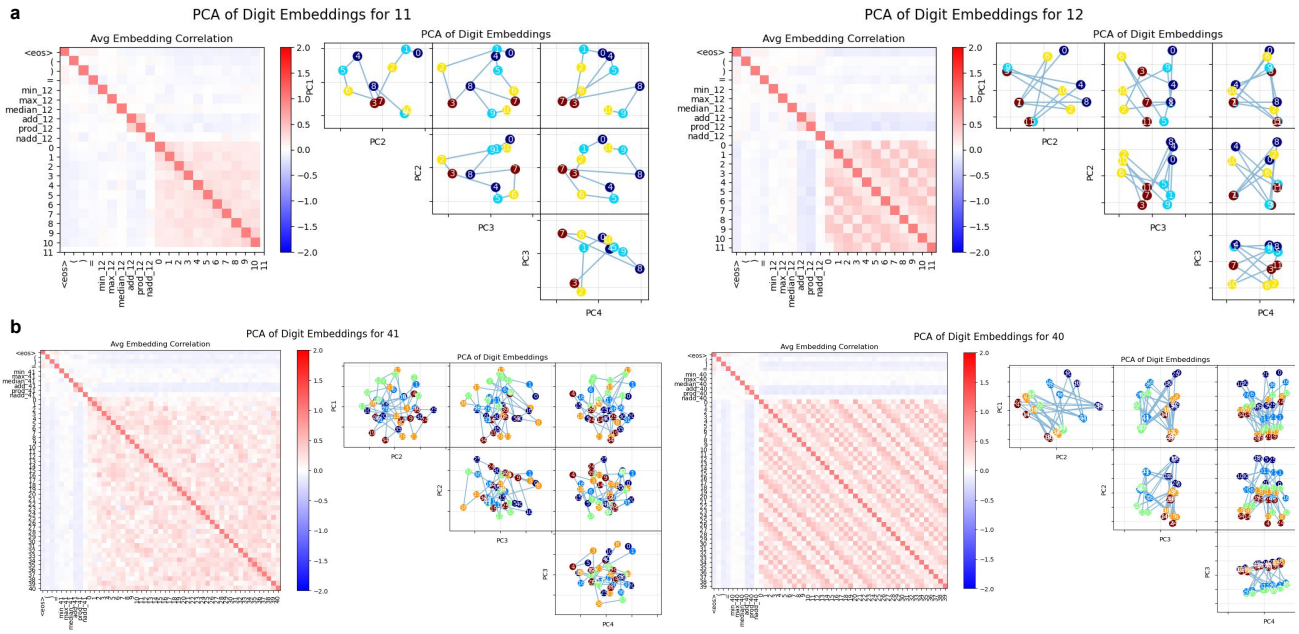


Figure 12. **PCs PROD modulo prime vs not prim.**

C. Permutation groups.

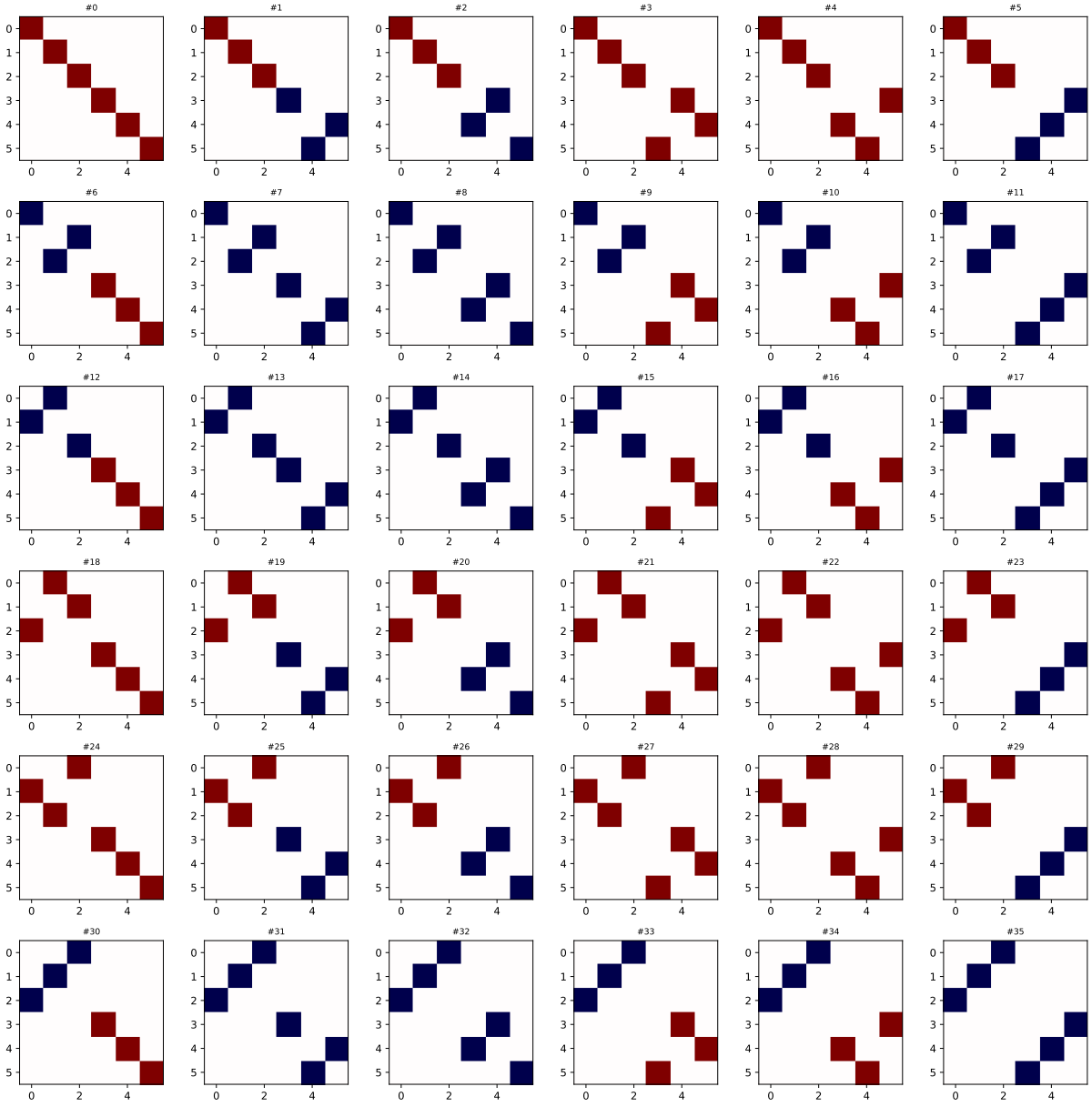


Figure 13. **Permutation - block diagonal with two 3x3 block matrix** Red indicates a cycle sub-block, while blue indicates a non-cycle block.

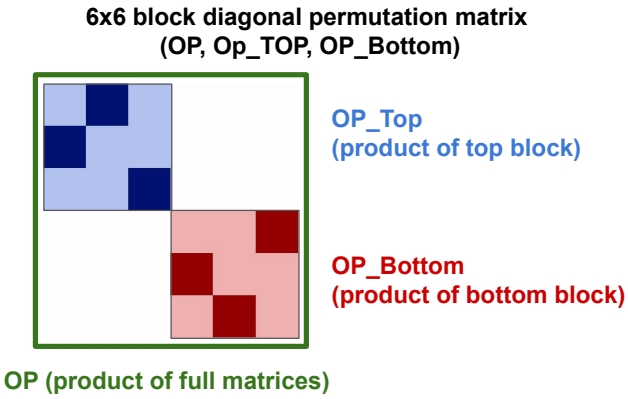


Figure 14. **6x6 block diagonal permutation operations(OP, Op_TOP, OP_Bottom)** OP_Top only acts on the top block matrix and OP_Bottom only acts on the bottom block matrix, while OP is an operation that acts on both the top and bottm blocks.

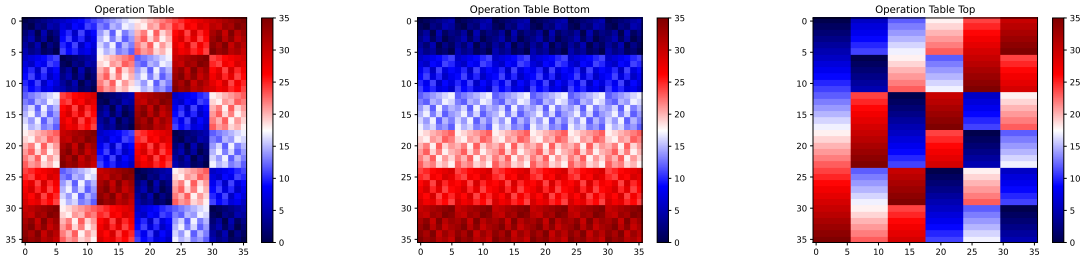


Figure 15. **Permutation data structure.** Left: Operation table (OP) - block diagonal with two 3x3 block matrix. Middle: Operation table (OP_BOTTOM) - block diagonal with two 3x3 block matrix. Right: Operation table (OP_TOP) - block diagonal with two 3x3 block matrix

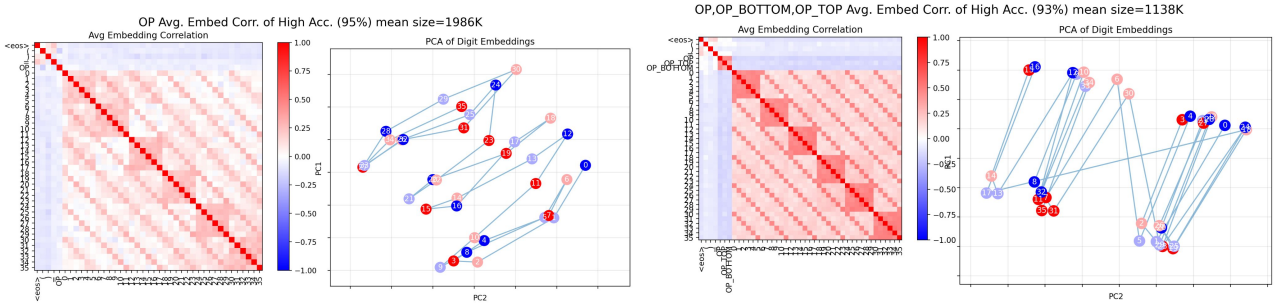


Figure 16. **Permutation experiments:** The correlation matrix and top 2 PCA embedding f=of permutaion matrices for high performing models in two settings: Left: trianing on the full 6×6 matrix multiplication (OP). Right: trianing on a mixture of OP, and subtasks OP_BOTTOM (multiplying bottom 3×3 block, repeating the top block of first matrix), and and OP_TOP (multiply top 3×3 block, repeat bottom of first matrix). The mixed task model reaches $> 90\%$ accuracy at 1.1k parameters, whereas the single OP model requires 1.9k for the same accuracy level.

D. Shuffled SUM

The Shuffled SUM table was constructed using the following code:

Listing 1. Generate upper triangle and diagonal matrices

```
# Generate upper triangle and diagonal matrices.
# To ensure commutativity i+j=j+i, we will transpose the
# the upper triangle.
upper_triangle_matrix = {(i, j): (i + j) % MOD
   for i in range(MOD)
   for j in range(MOD) if i < j}

# To ensure uniform distribution after shuffling the values,
# we must shuffle the diagonal separately.
# This is because all off-diag r.h.s. are repeated twice
# for commutativity, but not the diagonal entries.
diagonal_matrix = {(i, i): (2 * i) % MOD for i in range(MOD)}

# Function to shuffle values in a dictionary
def shuffle_dict_values(d):
   keys = list(d.keys())
   values = list(d.values())
   random.shuffle(values)
   return dict(zip(keys, values))

# Apply shuffling
shuffled_triangle = shuffle_dict_values(upper_triangle_matrix)
shuffled_diagonal = shuffle_dict_values(diagonal_matrix)
```

As a result, there is no consistent mapping between the original and shuffled values (e.g., the number 1 does not always map to 2), making it difficult for the model to learn a deterministic transformation. While the shuffled sum table remains commutative, we did not enforce associativity. A direct check also confirmed that it does not satisfy the associative property for most triplets.

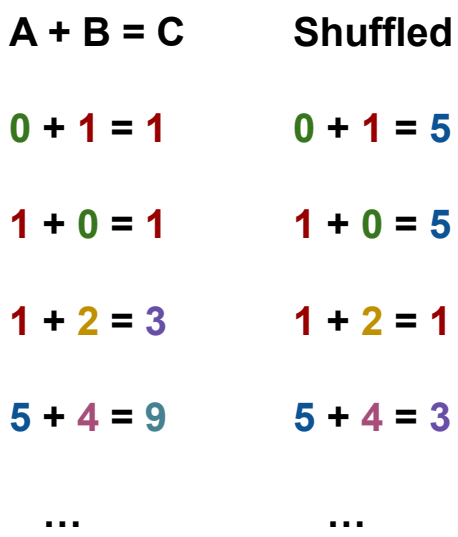


Figure 17. **Shuffle the summation table.**

D.1. Shuffled SUM modulo 10

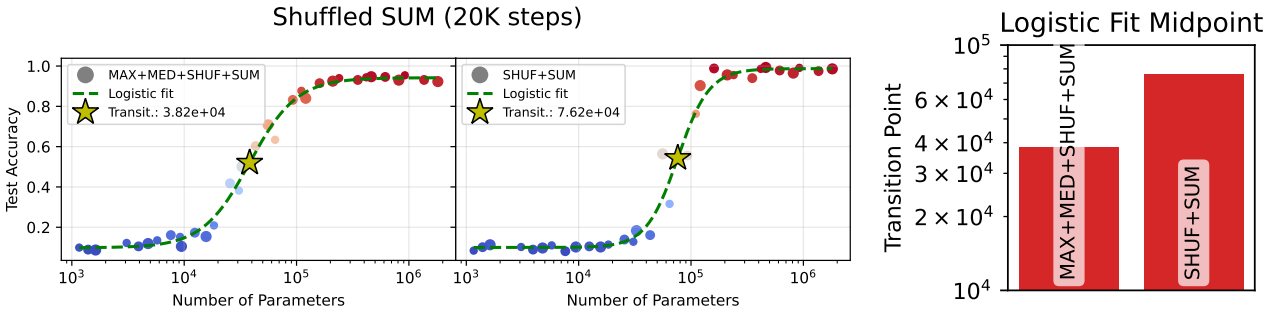


Figure 18. **Shuffled symmetric sum table, Mod 10:** We find that this shuffled version of sum is again more difficult to learn than any of the math operations except for the original Pure SUM, which remains slightly more difficult than even the shuffled version. Additionally, we observe that MAX+MED+Shuffled SUM is again more difficult than all operations except pure sum, suggesting that the number properties played an important role in the other tasks becoming easier in mixed training.

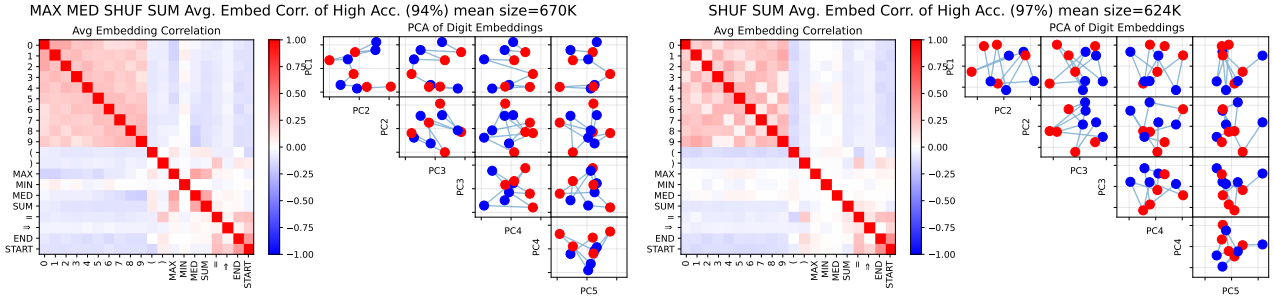


Figure 19. **PCA of embeddings mod 10 for Shuffled symmetric SUM, pure vs mixed with MAX and MED:** The numbers are colored based on parity (odd is red, even is blue). It is not expected that the embeddings show strong signals. There seems to be partial ordering of the numbers, but the clear wave patterns and clear parity separation is not evident.

D.2. Shuffled SUM modulo 26

We make a symmetric sum table ($A + B = B + A$), with a randomly shuffled right hand side, meaning where if in the table $A + B = C$, C does not the actual arithmetic sum of $A + B$ modulo 26. We do this to test a couple of hypotheses:

1. **Hypothesis 1:** The SUM trained alone is not really learning the logic arithmetic of numbers, but rather memorizing the sum table.
2. **Hypothesis 2:** Joint training with max and med leads to learning number properties.

If the first hypothesis holds, the shuffled sum would also require similarly high number of parameters as the normal sum and show similarly random patterns in the embedding space. If the second hypothesis holds, then joint training of shuffled sum should actually have detrimental effects on learning sum because number properties don't play a role in learning shuffled sum. Thus we expect the jointly trained model to struggle to learn all three operations, or require significantly higher number of parameters to master shuffled sum, compared to joint training on regular sum.

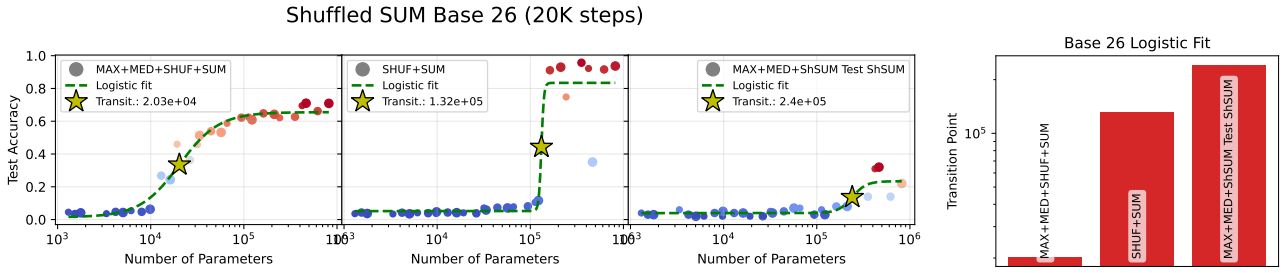


Figure 20. **Shuffled symmetric sum table, Mod 26, 20k steps:** We find that this shuffled version of sum is again more difficult to learn than any of the math operations except for the original Pure SUM, which remains slightly more difficult than even the shuffled version. We also observe that MAX+MED+Shuffled SUM never reaches more than 80% accuracy. The third scatter plot from the let shows the accuracy of the MAX+MED+Shuffled SUM model on the Shuffled SUM test set. We see that the accuracy is very low ($\sim 20\%$ top), showing that the mixed model never learned the shuffled SUM. This may suggest that MAX+MED revealed number properties, but Shuffled SUM was incompatible with those properties, leading to a model that overall cannot solve the two problems (MAX+MED and Shuffled SUM) simultaneously.

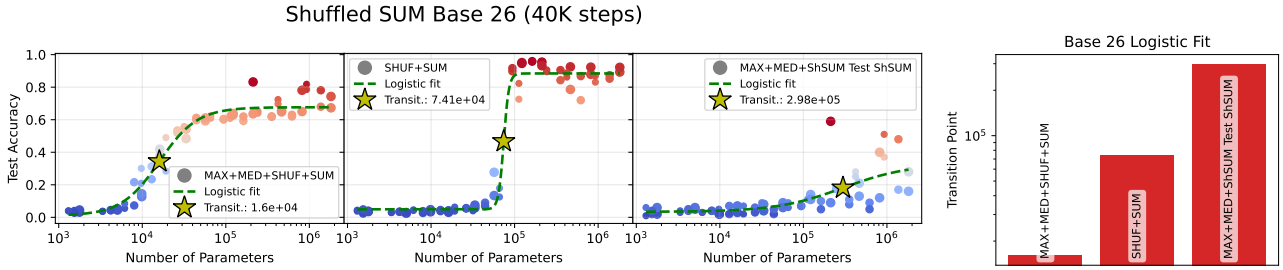


Figure 21. **Shuffled symmetric sum table, Mod 26, 40 steps:** We find that this shuffled version of sum is again more difficult to learn than any of the math operations except for the original Pure SUM, which remains slightly more difficult than even the shuffled version. We also observe that MAX+MED+Shuffled SUM never reaches more than 80% accuracy. The third scatter plot from the let shows the accuracy of the MAX+MED+Shuffled SUM model on the Shuffled SUM test set. We see that the accuracy is very low ($\sim 20\%$ top), showing that the mixed model never learned the shuffled SUM. This may suggest that MAX+MED revealed number properties, but Shuffled SUM was incompatible with those properties, leading to a model that overall cannot solve the two problems (MAX+MED and Shuffled SUM) simultaneously.

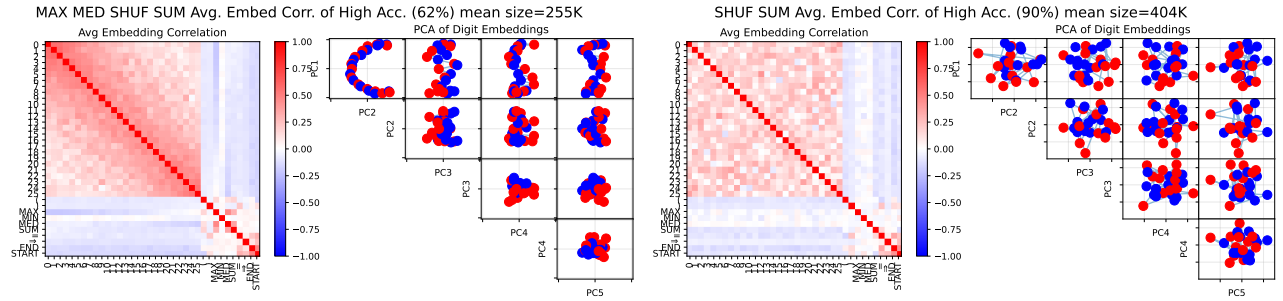


Figure 22. **PCA of embeddings for Shuffled symmetric SUM, pure vs mixed with MAX and MED, Mod 26:** The numbers are colored based on parity (odd is red, even is blue). There seems to be partial ordering of the numbers, but the clear wave patterns are not visible. We do almost observe parity separation in PC3 and PC4, albeit with some noise. It is curious that the system still learns partial parity, but evidently this feature did not allow the system to learn the shuffled SUM perfectly.

E. Deep transformer vs. Recurrent Transformer

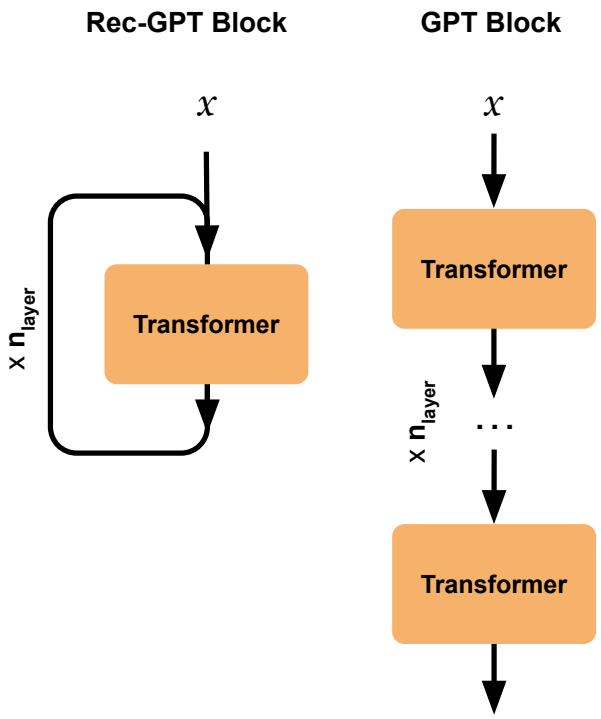


Figure 23. **Recurrent Transformer vs. Deep transformer** In the Rec-GPT block, there is only one transformer block that is applied recursively n times, so each output becomes the input for the next step. In contrast, in a traditional GPT block, there are n transformer blocks stacked sequentially on top of each other.

F. Results using Deep Transformer based models

F.1. Experiment settings

1. **Model**: GPT model ((Karpathy, 2022))
2. **Number of layers**: 1, 2, 3, 4, 5, 6
3. **Number of head**: 1
4. **Embedding dimension**: 4, 8, 12, 16, 24, 32, 48, 64, 96, 128, 192, 256
5. **Number of random seed**: 5
6. **Context window size**: 128
7. **Batch size**: 64
8. **Optimizer**: Adam
9. **Minimum learning rate**: 1e-4
10. **Maximum interation steps**: 20000/50000
11. **Early Stopping criteria**: Early stopping was applied after 2000 iterations when the change in training loss was below Δ_{min} = 2.5e-4 for 10 consecutive evaluation steps.

12. **Data** We generate 50,000 initial equations and split them into training and test sets by excluding 100 randomly selected triplets. The training set consists of approximately 45,000 examples that do not contain any of the excluded triplets. The test set comprises around 2,000 examples, carefully curated to ensure that the excluded triplets do not appear, even in the final step of each equation. We evaluate model performance on a subset of 1,000 final test examples.

13. **Vocab - Base 10 ListOps**: `%()+-/0123456789=>es`

14. **Vocab - Base 26 ListOps**: `ABCDEFGHIJKLMNOPQRSTUVWXYZse()+-/%=>`

15. **Hardware**: All simulations were run on a mix of GPUs, including NVIDIA A100, H200, RTX 4090, and A30, as well as on a standard modern laptop. The experiments are lightweight, requiring approximately 1.5GB of RAM, and can be executed efficiently on any recent laptop-class device.

F.2. Experiments on ListOps.

1. **Base 10**: - no CoT/CoT: MAX, MED, SUM, MAX/MED/SUM

2. **Base 10**: - all combination: MAX, MIN, MED, SUM + MAX/MED/SUM-RANDOM SHUFFLED

3. **Base 26**: - all combination: MAX, MIN, MED, SUM + MAX/MED/SUM-RANDOM SHUFFLED

4. **Triplet**: MAX+MED+SUM 2700 (2700 training data samples = 900 MAX + 900 MED + 900 SUM), MAX+MED+SUM 900 (900 training data samples = 300 MAX + 300 MED + 300 SUM), SUM (900 training data samples)

F.3. Data and processing

F.3.1. Dataset Notation

ListOps consists of nested mathematical equations involving operations such as min, max, median, and sum modulo 10 applied to single-digit numbers (0-9). It uses the Polish notation: (operation, [inputs]) For example: `max(3,min(7,4,9))=4`.

`max(3,min(7,4,9))=4` ⇒ **Polish:** `(max,3,(min,7,4,9))=4`

To disentangle any complexity arising from tokenization we further simplify these expression by representing the by symbols: '$+$' for max, '$-$' for min, '$/$' for median, and '$\%$' for sum modulo 10. For example:

`(max,3,(min,7,4,9))=4` ⇒ **Our notation:** `s(+3(-749))=4e`

In this notation, 's' denotes the start of the expression, 'e' marks the end, and parentheses indicate nesting levels.

F.3.2. Tokenization

We employ word-level tokenization where each semantically meaningful unit is a single token:

- **Operations:** Each operation with its modulus is one token (e.g., `add_20`, `max_10`, `prod_19`).
- **Numbers:** Each number is one token, regardless of digit count (e.g., `3`, `13`, `104` are each single tokens).
- **Symbols:** Parentheses `(`, `)`, equals =, CoT separator >, and end token `<eos>` are each single tokens.

Note that commas are not needed as token boundaries are implicit. For modulo n, the vocabulary size is approximately n + (number of operations) + 5 (for symbols). This tokenization ensures that each number is atomic, preventing the model from learning spurious digit-level patterns.

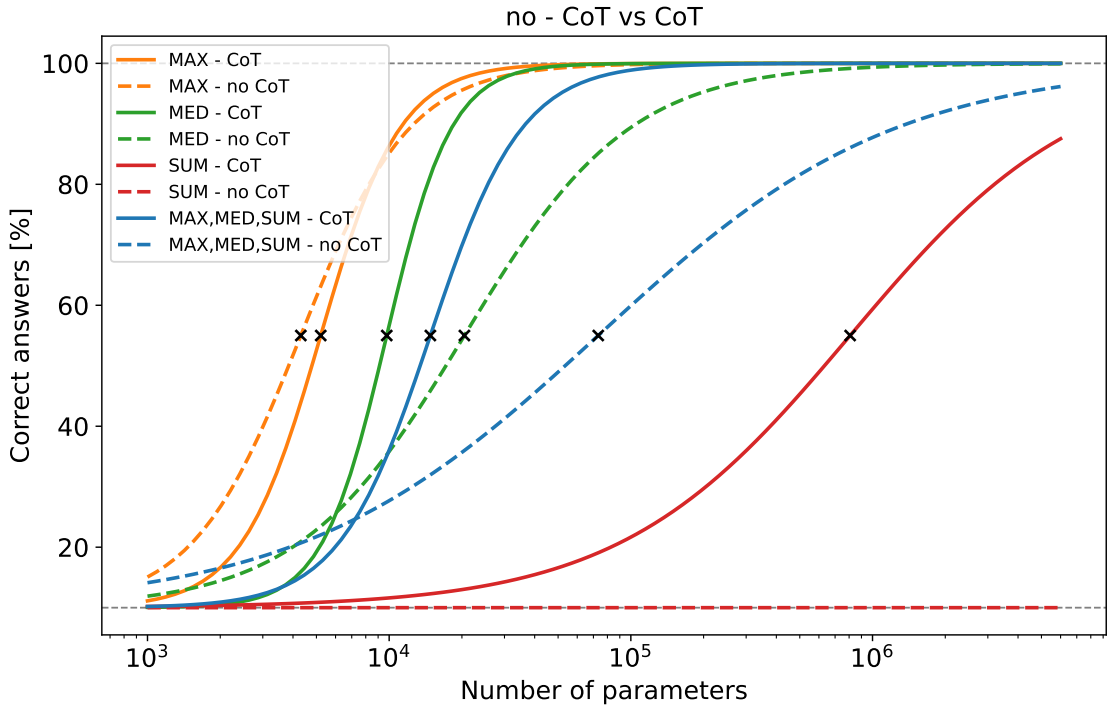


Figure 24. **no - CoT vs. CoT.** Providing solutions as chain-of-thought (CoT) helps models learn the tasks. In almost all cases, CoT accelerates learning, enabling smaller models to succeed. This effect is especially strong for the sum operation, which cannot be learned without CoT.

F.3.3. Chain of Thought Implementation

Directly solving nested ListOps in one step is challenging for transformers (Fig. 24). Even with maximum three nesting levels and three operands, GPT models with over 10 million parameters fail to learn. To address this, we use a chain-of-thought (CoT) approach that resolves the innermost operation at each step.

For example, with the functional notation:

```
add_10(1 2 add_10(3 4))>add_10(1 2 7)>0=0<eos>
```

The CoT proceeds as:

- Initial expression: `add_10(1 2 add_10(3 4))`
- Step 1: Resolve innermost `add_10(3 4)` → `7`
- Step 2: Resolve `add_10(1 2 7)` → `0`
- Final: `0=0<eos>`

This CoT approach guides the model through step-by-step reasoning, provides granular supervision, and significantly improves performance on challenging operations (Fig. 24).

F.4. Modulo 26

We define the following token vocabulary `ABCDEFGHIJKLMNOPQRSTUVWXYZse()+-/%=>`, where letters are mapped to integers such that `A` → 0, `B` → 1, ..., `Z` → 25.

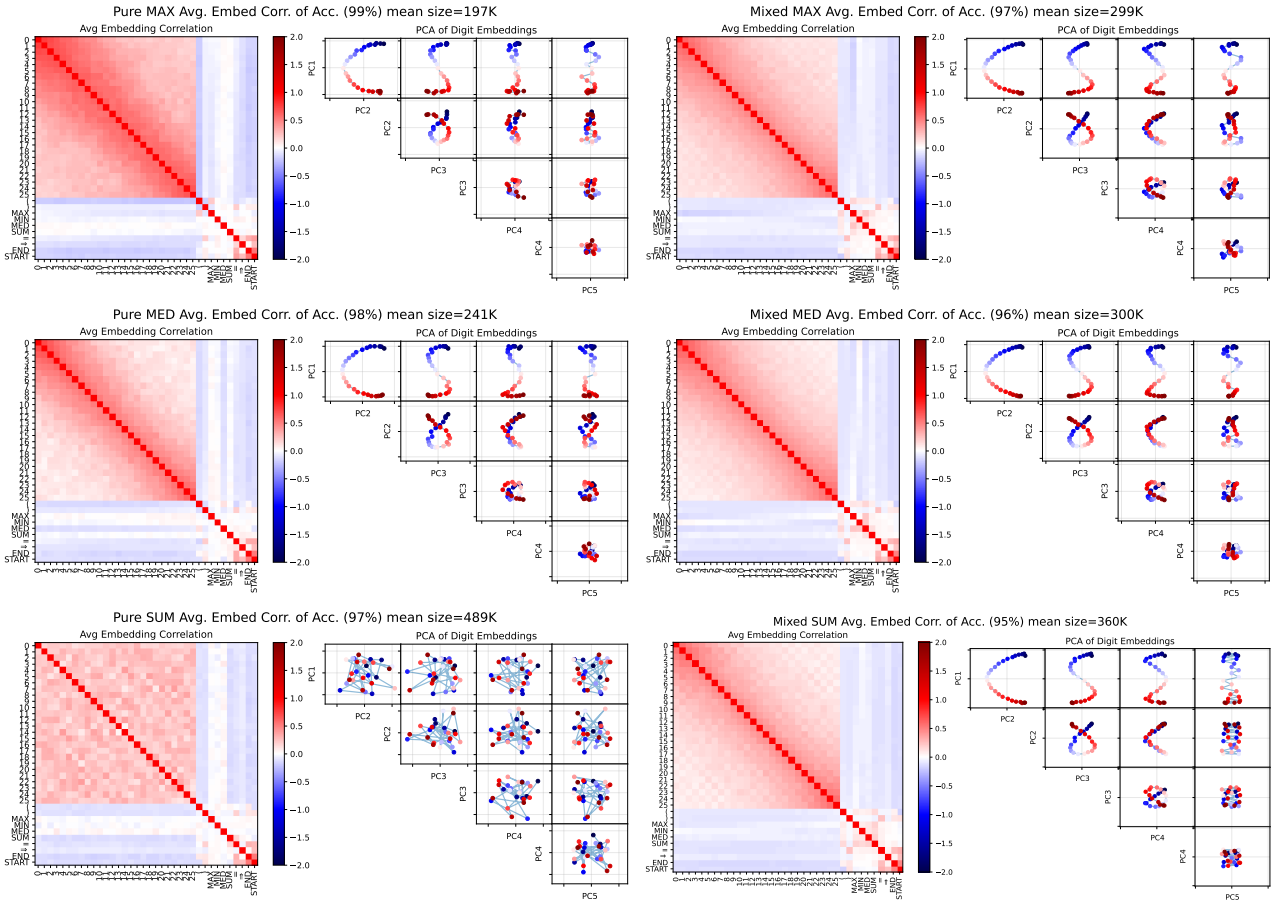


Figure 25. **PCA of embeddings:** We choose all models which reached over 95% test accuracy. Each row shows the average correlation matrix and top PCs for models trained on either a single operation, e.g. Pure+MAX, or all mixtures involving a given operation, e.g. Mixed+MAX. Again, pure SUM does not show a discernible structure in the embeddings, whereas all cases do.

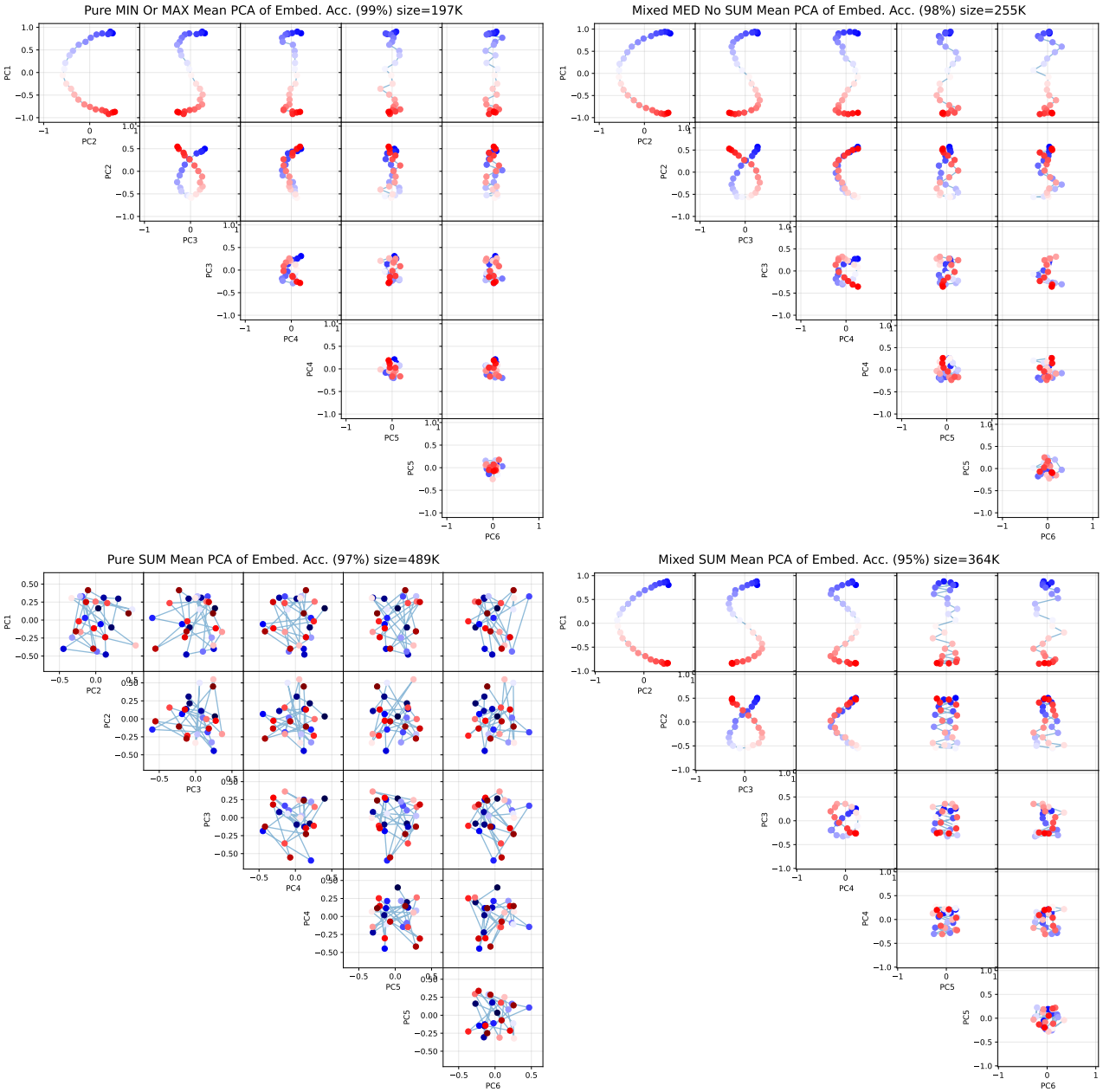


Figure 26. **PCA of embeddings:** We choose all models which reached over 95% test accuracy. Each row shows the average correlation matrix and top PCs for models trained on either a single operation, e.g. Pure+MAX, or all mixtures involving a given operation, e.g. Mixed+MAX. Again, pure SUM does not show a discernible structure in the embeddings, whereas all cases do.

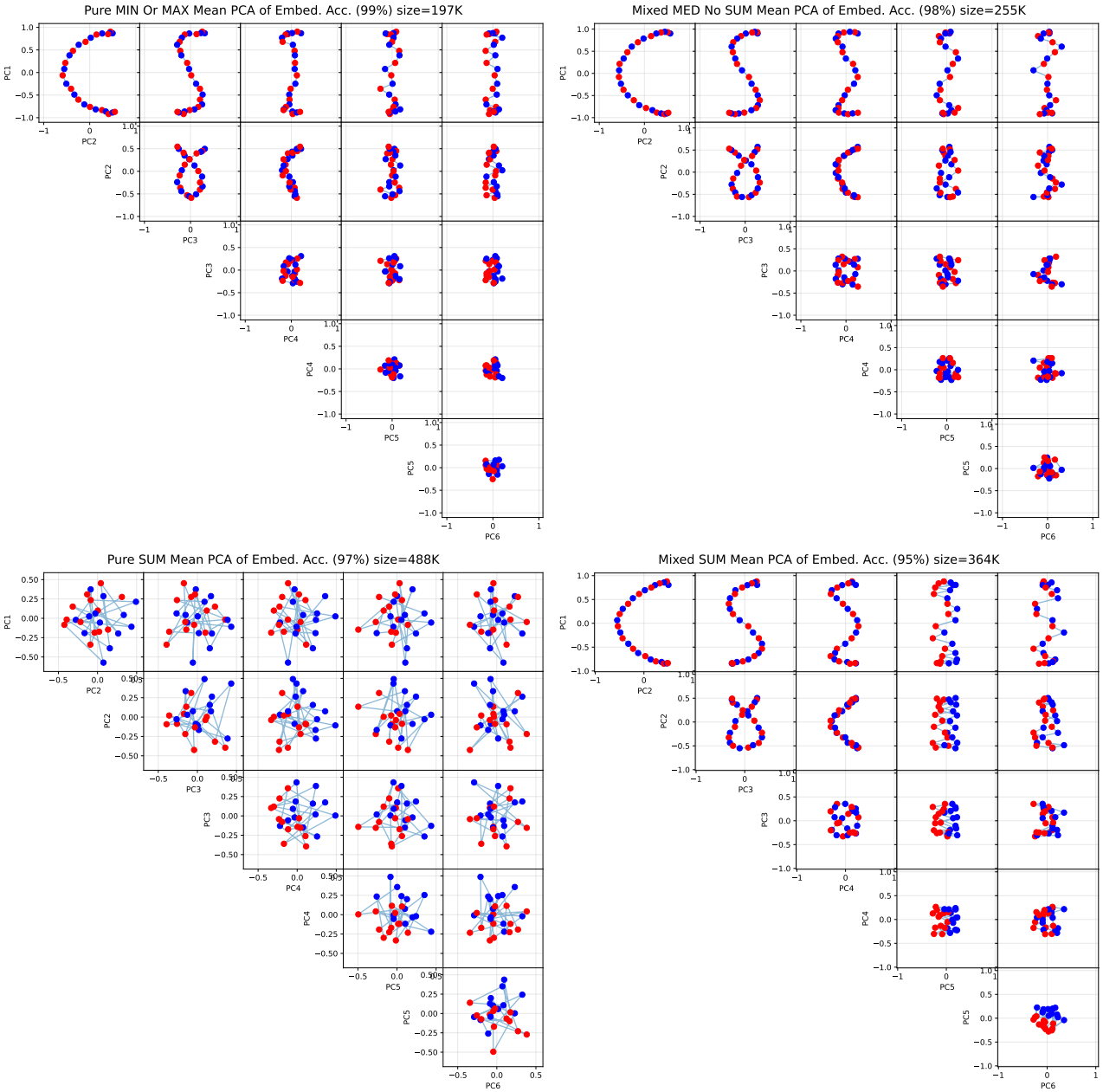


Figure 27. **PCA of embeddings Odd vs Even:** Same plot as above, only odd numbers colored red and even colored blue. Mixed SUM shows a clear odd-even separation in a few of the top PCs. Such a separation is not clearly observed in other cases. Interestingly, Pure SUM approximately separates odd-even, suggesting such separation may play a role in its algorithm.

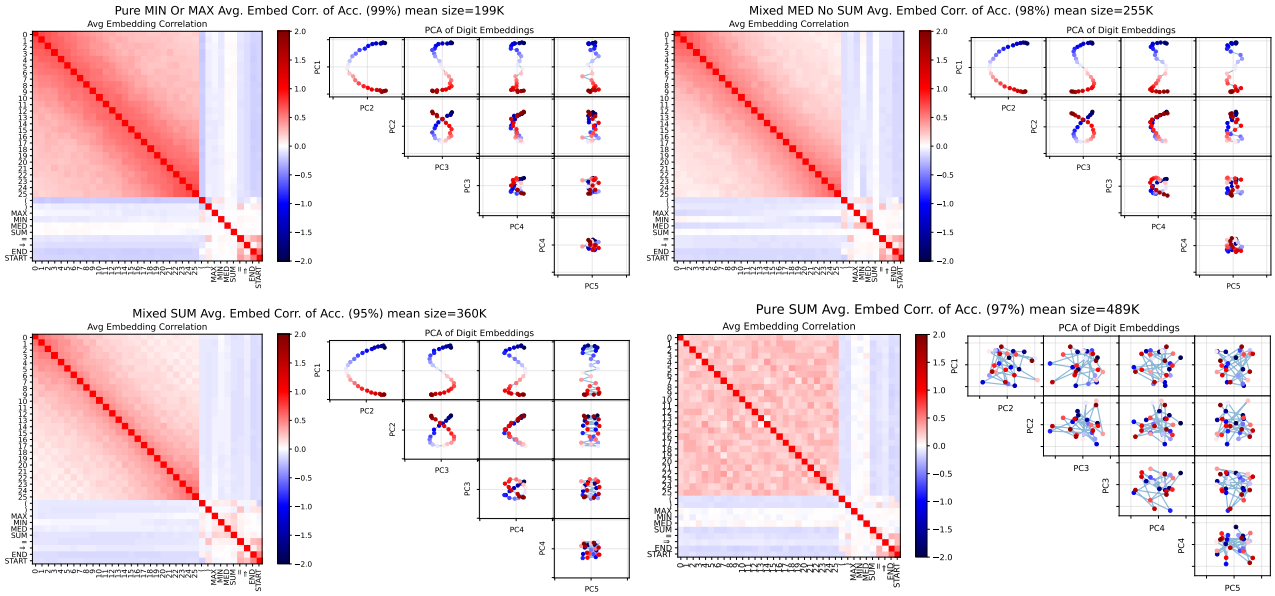


Figure 28. **PCA of embeddings:** We choose all models which reached over 90% test accuracy. Each row shows the average correlation matrix and top PCs for models trained on either a single operation, e.g. Pure MAX, or all mixtures involving a given operation, e.g. Mixed SUM. Interestingly, pure SUM does not show a discernible structure in the embeddings, whereas all other cases do. Notably, Mixed SUM models exhibit a prominent odd-even separation in PC5.

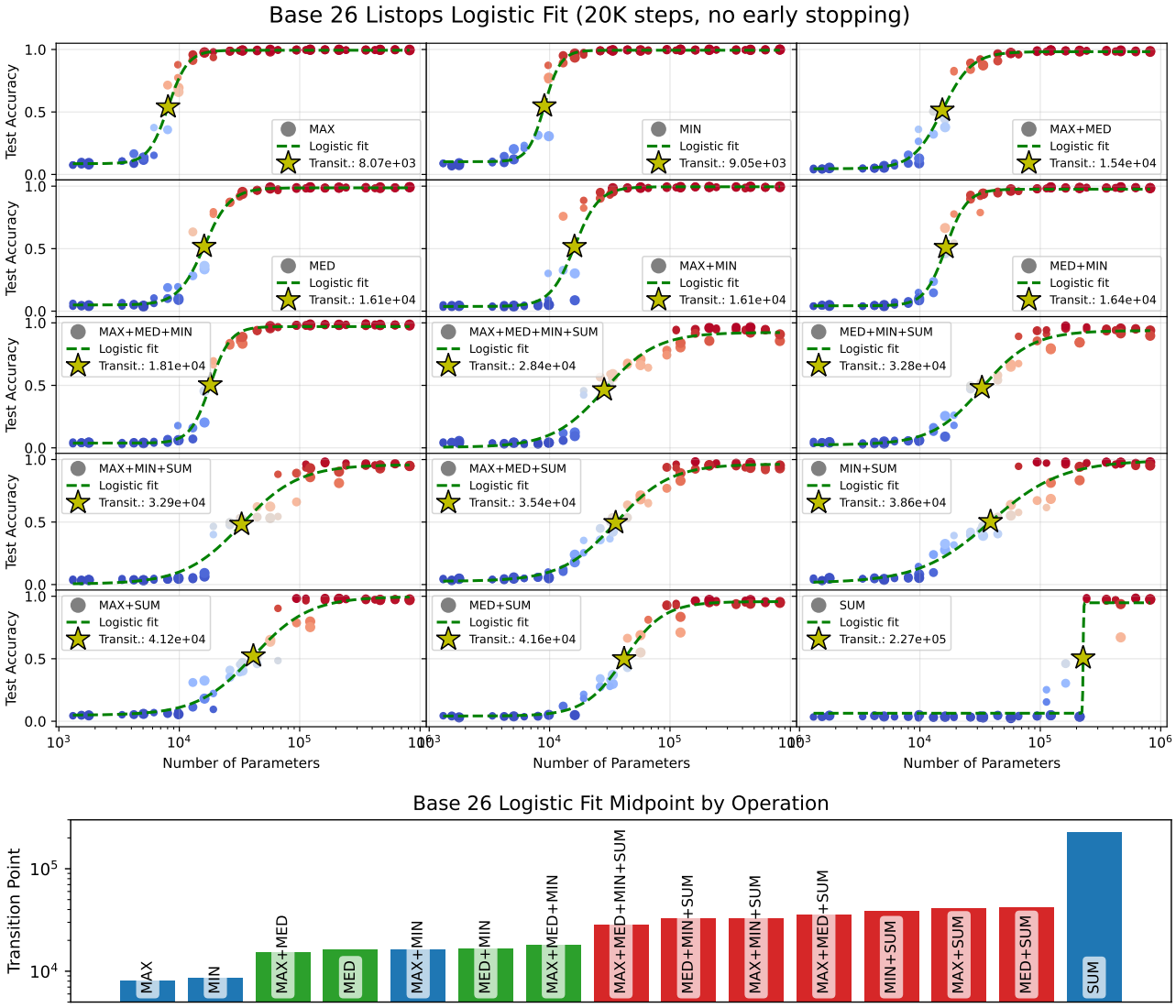


Figure 29. **Emergence of abilities in ListOps:** Each plot shows the same group of small transformer models trained on a different mix of the four operations MAX, MIN, MED, and SUM. Red dots are models reaching more than 50% accuracy, and blue dots are less than 50%. The dashed green line is a logistic fit, and the yellow star indicates the transition point at 50%. The x-axis is the model size (number of parameters), and the plots are sorted in ascending order of transition points. The bottom panel shows a bar plot of the model sizes at the transition points, with each group distinguished by a different color.

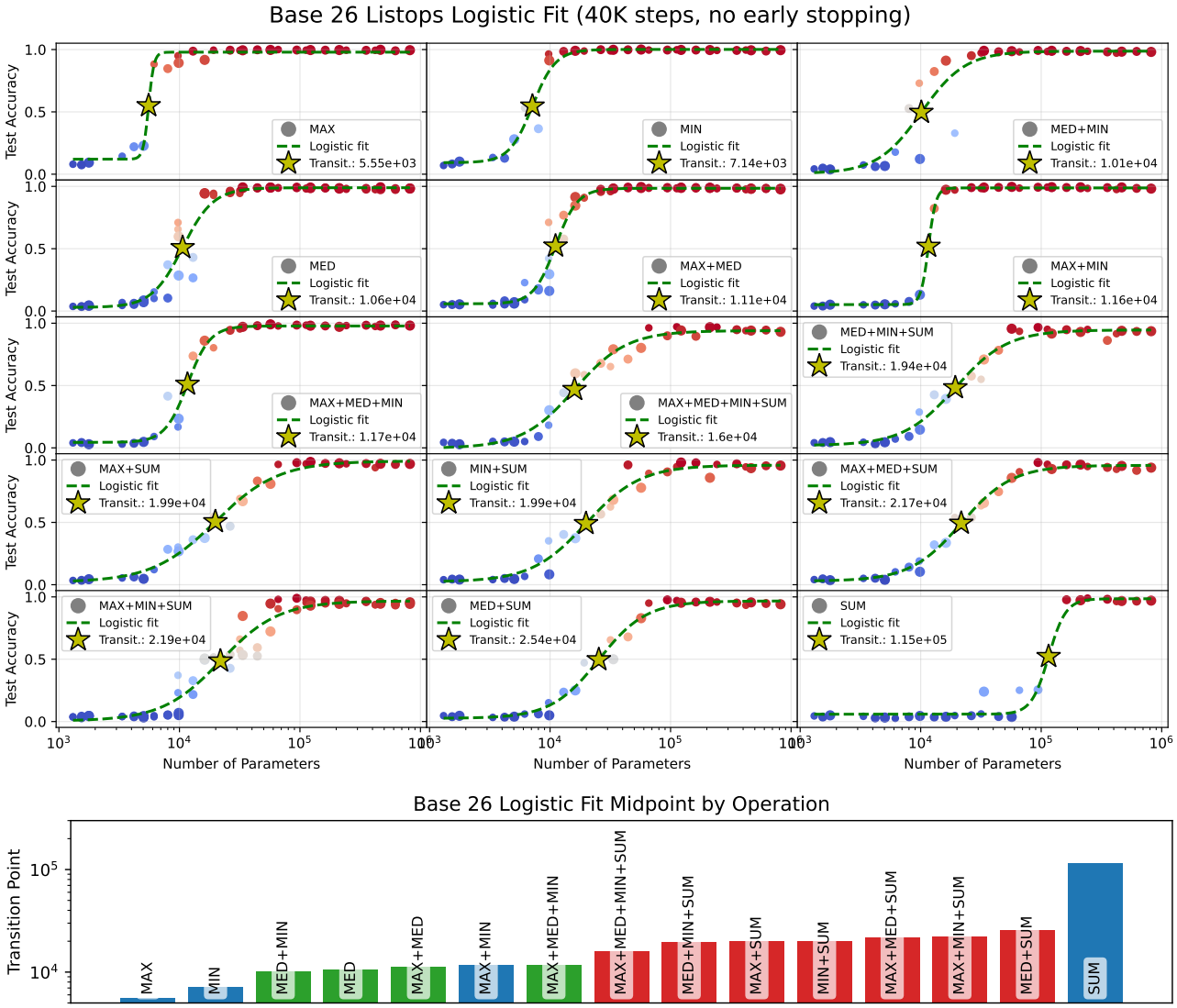


Figure 30. **Emergence of abilities in ListOps, 40k steps:** Long training. Each plot shows the same group of small transformer models trained on a different mix of the four operations MAX, MIN, MED, and SUM. Red dots are models reaching more than 50% accuracy, and blue dots are less than 50%. The dashed green line is a logistic fit, and the yellow star indicates the transition point at 50%. The x-axis is the model size (number of parameters), and the plots are sorted in ascending order of transition points. The bottom panel shows a bar plot of the model sizes at the transition points, with each group distinguished by a different color.

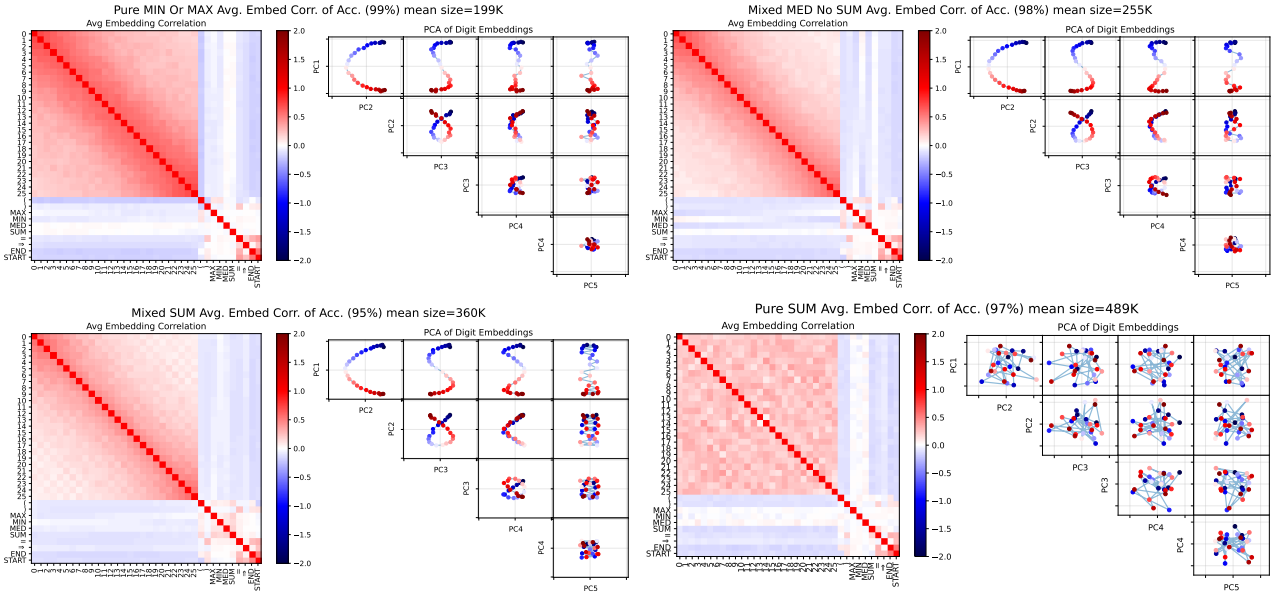


Figure 31. **PCA of embeddings:** We choose all models which reached over 90% test accuracy. Each row shows the average correlation matrix and top PCs for models trained on either a single operation, e.g. Pure MAX, or all mixtures involving a given operation, e.g. Mixed SUM. Interestingly, pure SUM does not show a discernible structure in the embeddings, whereas all other cases do. Notably, Mixed SUM models exhibit a prominent odd-even separation in PC5.

F.5. Modulo 10

We conducted the same experiments also on mod 10. The smaller number of numbers makes definitive statements about some of the patterns more challenging. But all the patterns we observed in mod 26 also have parallels in mod 10, including the prominent odd-even split for operations involving SUM. Token vocabulary: `%()+-/0123456789=>es`

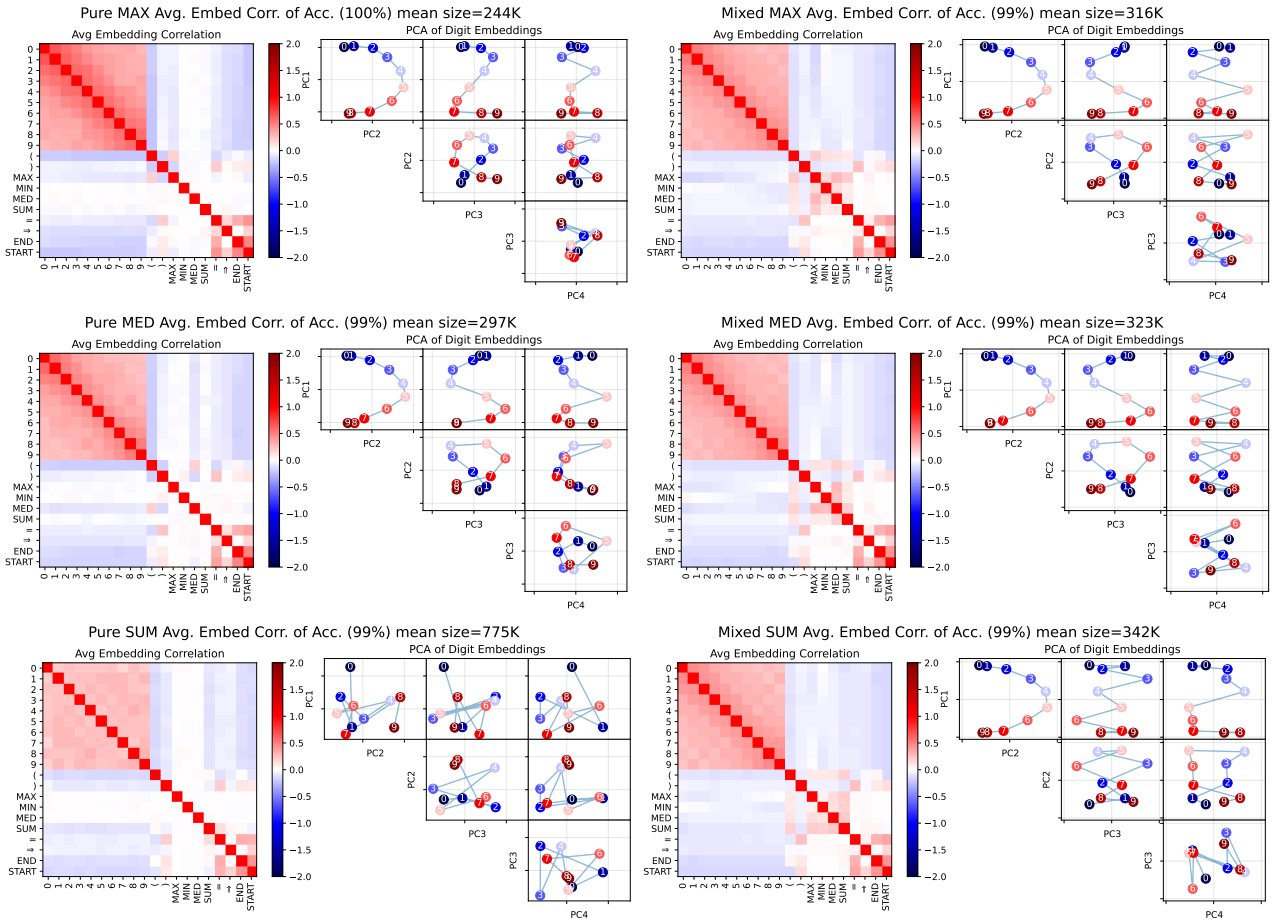


Figure 32. **PCA of embeddings:** We choose all models which reached over 95% test accuracy. Each row shows the average correlation matrix and top PCs for models trained on either a single operation, e.g. Pure+MAX, or all mixtures involving a given operation, e.g. Mixed+MAX. Again, pure SUM does not show a discernible structure in the embeddings, whereas all cases do.

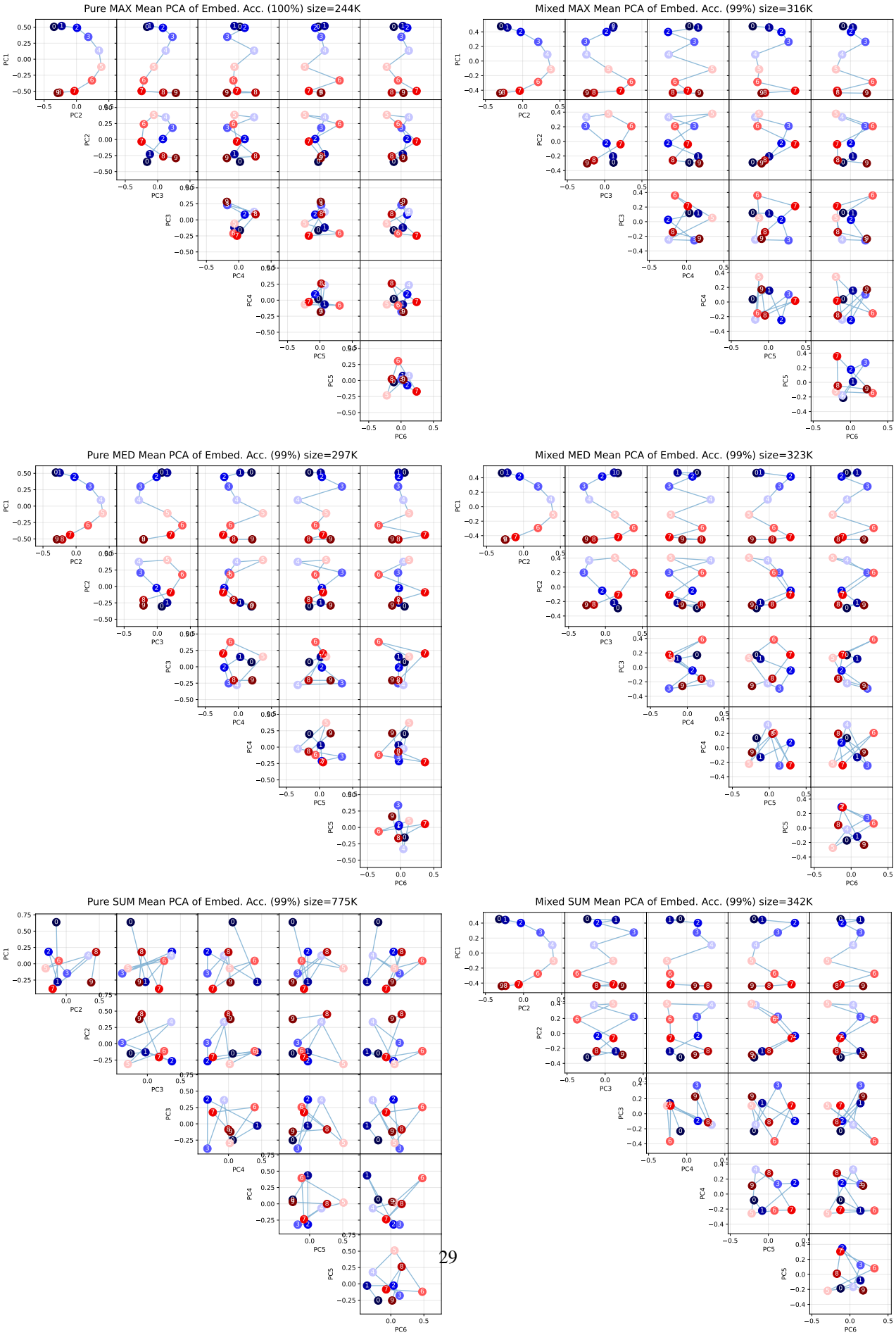


Figure 33. **PCA of embeddings:** We choose all models which reached over 95% test accuracy. Each row shows the average correlation

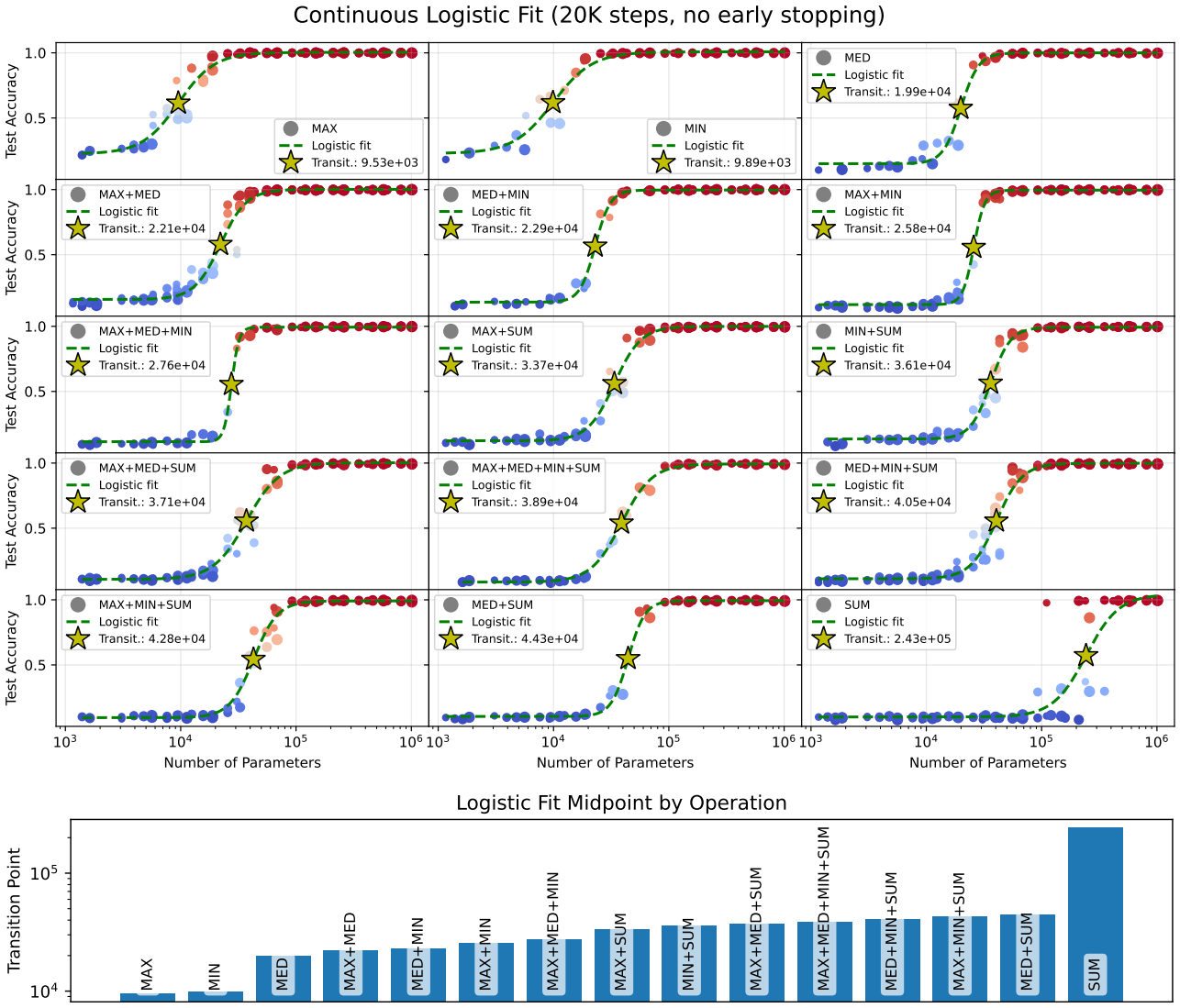


Figure 34. **Emergence of of abilities in ListOps, Mod 10:** Each plot shows the same group of small transformer models trained on different variants of ListOps. Each variant uses a different mix of the four operations MAX, MIN, MED, SUM. Red dots are model reaching more than 50% accuracy, and blue are less than 50%. The dashed green line is a logistic fit and the yellow star indicates the transition point at 50%. The x axis is the model size and the plots are sorted in ascending order of transition points. The bottom is a bar plot showing the model size at the transition point. We observe that SUM is a clear outlier, with models requiring significantly more parameters to learn SUM. Surprisingly, combining SUM with other operations dramatically reduces the transition point, with model less than half the size easily reaching 100% accuracy.

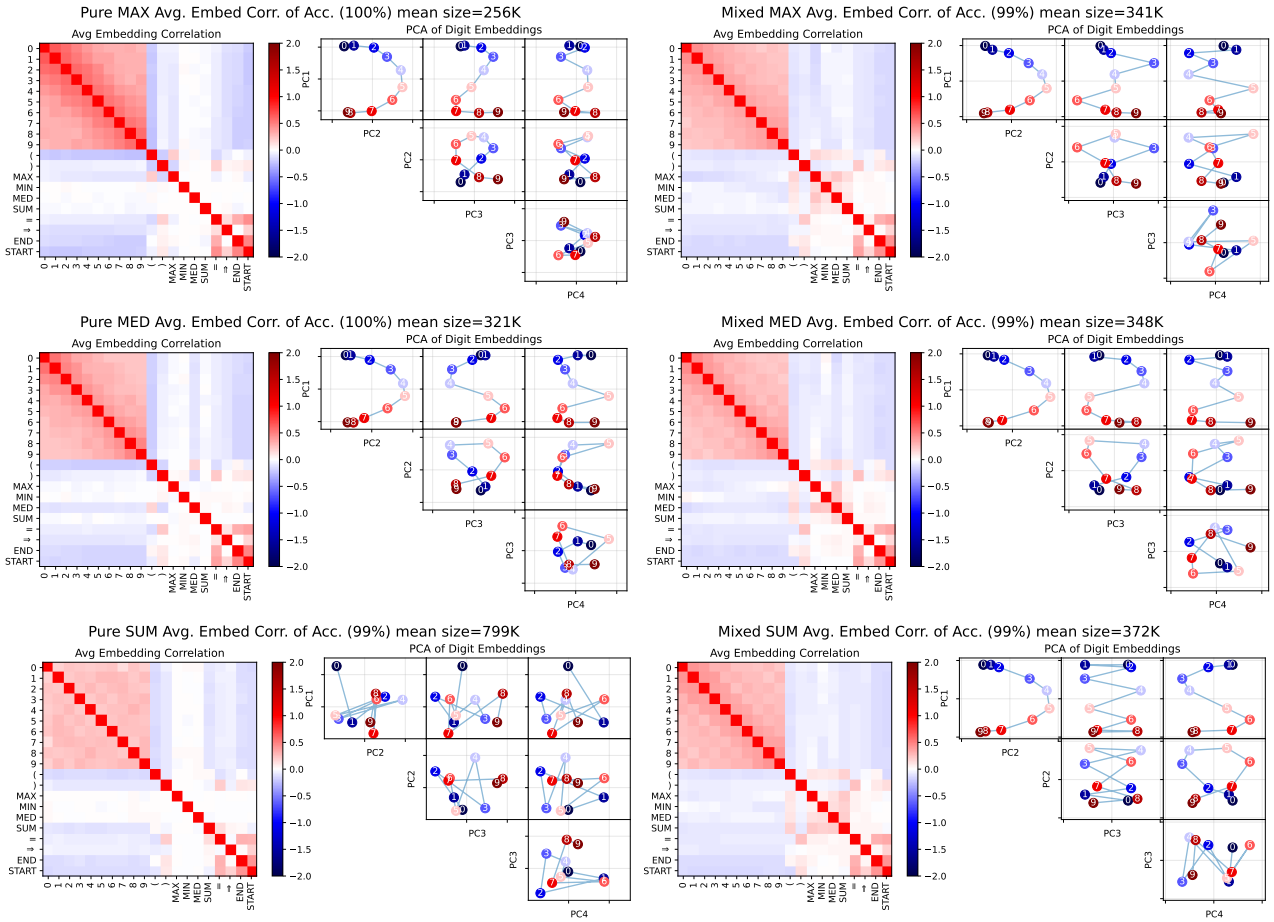


Figure 35. **PCA of embeddings with the same sample size (67):** We choose the smallest 67 models which reached over 95% test accuracy. Each row shows the average correlation matrix and top PCs for models trained on either a single operation, e.g. Pure+MAX, or all mixtures involving a given operation, e.g. Mixed+MAX. We observe that the PCs get slightly distorted with the smaller sample size compared to fig. 33, but the overall structure stays the same. Again, pure SUM does not show a discernible structure in the embeddings, whereas all cases do.

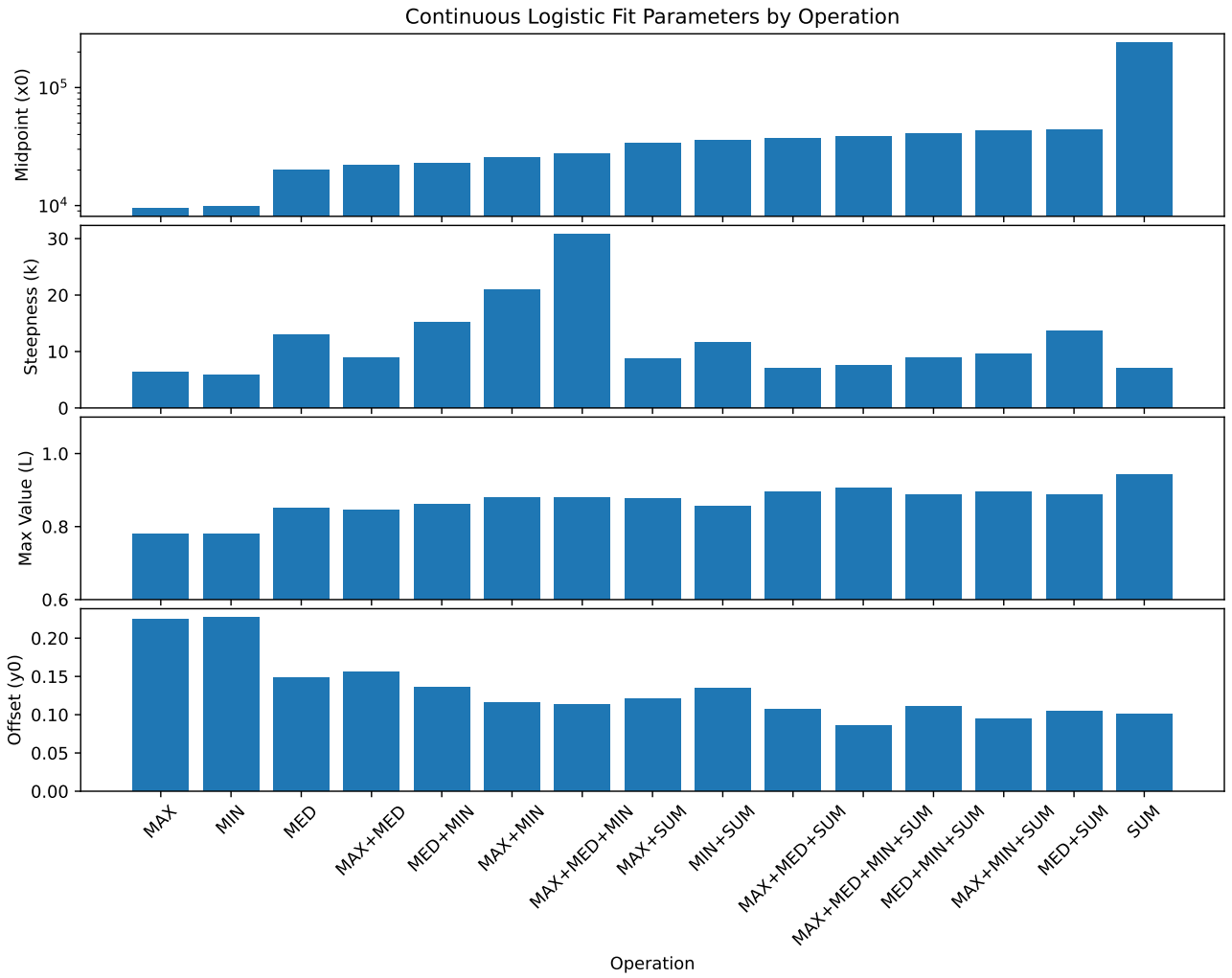


Figure 36. **Parameters of continuous logistic regression fitting accuracy to number of parameters (No Early stopping).**

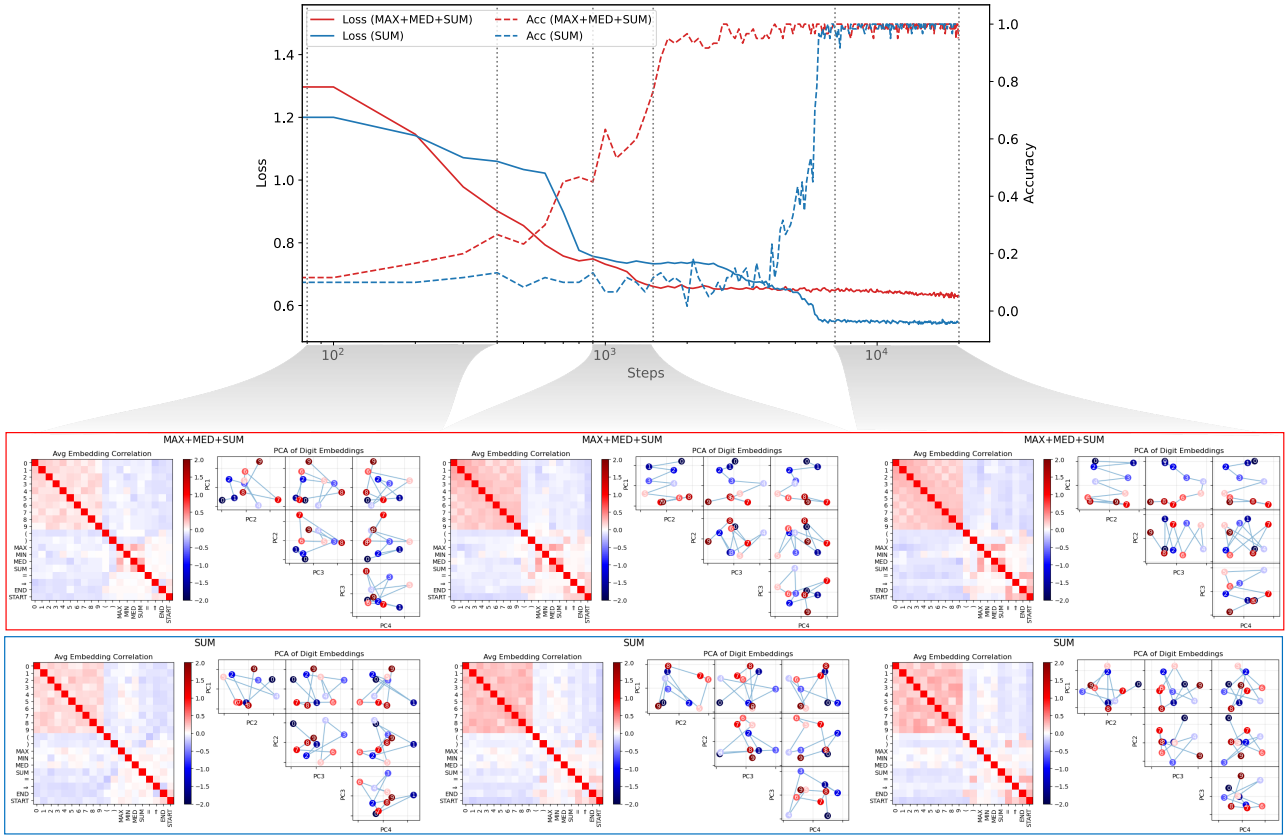


Figure 37. **Evolution of training loss, accuracy, and principal components of cosine similarities in the embedding layer. Modulo 10 ListOps.** The top main figure shows the evolution of training loss (solid lines) and test accuracy (dashed lines) for models with an embedding dimension of 128 and 3 layers, trained either on SUM-only data (blue) or on mixed MAX+MED+SUM data (red). Curves represent the mean across three independent runs, with shaded regions indicating one standard deviation. All models were trained for 20000 iterations. The red and blue boxes beneath the main plot display the average embedding representations at different training stages (indicated by vertical gray dashed lines). PCA reveals that models trained on MAX+MED+SUM data progressively develop a structured representation of numerical concepts, accompanied by a steady decrease in loss. In contrast, models trained solely on SUM data exhibit no clear structure in the embedding space and show long plateaus in the loss curve. This suggests that discovering an effective algorithm for the SUM operation in isolation requires significantly more exploration during training, in contrast to the more efficient joint training setting.

F.6. Triplet Experiments

To increase control in our experimental setup, we train models only on triplet inputs 38. We construct 1000 unique triplet samples, splitting them into 900 for training and 100 for testing.

For joint training on the MAX, MED, and SUM operations, we use a combined training set of 2700 triplets (900 per operation) and evaluate performance on the same 100 excluded SUM samples. For a fair comparison, we also create a balanced training set of 900 samples by selecting 300 examples from each of MAX, MED, and SUM.

With early stopping, models trained on the full 2700-sample mixed dataset achieve 100% accuracy on the SUM test set, while those trained on only 900 SUM samples reach approximately 50% accuracy. When trained for 50k steps without early stopping, models eventually learn the task, but SUM requires significantly more parameters—consistent with earlier observations. These results highlight that learning the SUM operation in isolation is more challenging and slower (Fig. 38).

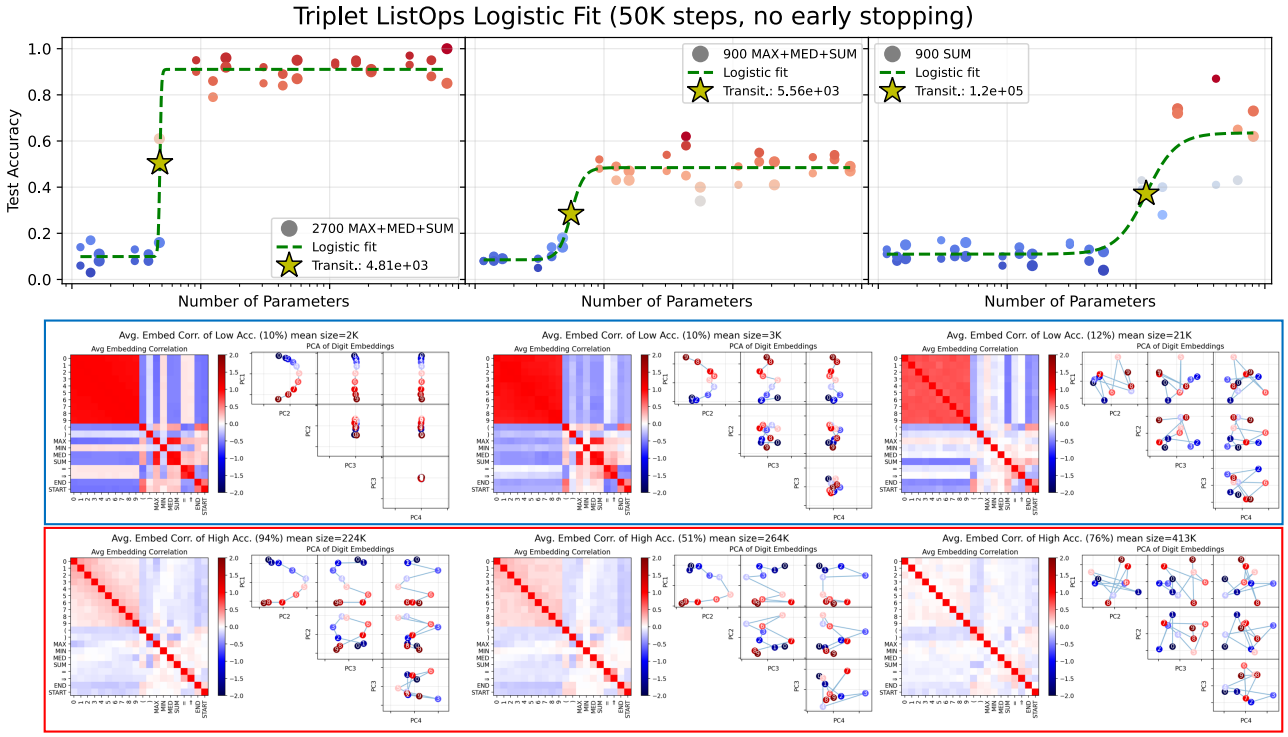


Figure 38. **Training only on triplet ListOps data.** The first row shows the same group of small transformer models trained on different variants of the triplet dataset. Each training set is constructed from 900 unique triplets. The 2700 MAX+MED+SUM dataset includes all 900 triplets, each labeled with three different operations. The 900 MAX+MED+SUM dataset contains 300 examples per operation, randomly sampled from the 2700 set. The 900 SUM dataset contains only the 900 unique SUM triplets. In all cases, the test set comprises the same held-out 100 SUM triplets. Red dots indicate models that exceed 50% test accuracy, while blue dots denote models that fall below this threshold. The dashed green line represents a logistic fit, and the yellow star marks the transition point at 50% accuracy. The x-axis shows the model size (in number of parameters), and subplots are ordered by increasing transition threshold. The red and blue boxes beneath the main plot display the average PCA embedding representations at selected regions; red corresponds to high-accuracy models, while blue indicates low-accuracy ones. Models trained on the combined MAX+MED+SUM task develop a discernible structure during training, unlike models trained solely on SUM. Notably, augmenting MAX and MED data with only 300 SUM samples allows the model to achieve 50% accuracy on a held-out 100 SUM sample test set, using a model 10× smaller than those trained on 900 SUM samples alone, which only reach $\approx 80\%$ accuracy.

G. Observations from the norm of Attention and Feedforward outputs

To investigate the internal dynamics of our models, we focused on the final layer of a 3-layer transformer network, featuring a single attention head and an embedding dimension of 128. Our analysis centered on comparing the behavior of models trained on "ALL3" operations (MAX+MED+SUM) versus those trained solely on the "SUM" operation.

We introduced a novel metric to quantify the impact of different components within the network: the ratio of output to input norms for both the self-attention (SA) and feedforward (FFN) sublayers. Specifically, we computed:

1. Attention ratio: $r_{attn} = \frac{\|SA(LN_1(x))\|}{\|x\|}$
2. Feedforward ratio: $r_{ffwd} = \frac{\|FFN(LN_2(x_1))\|}{\|x_1\|}$

where LN_1 and LN_2 are layer normalization operations, and x_1 is the output of the self-attention sublayer. These ratios provide insight into how much each component modifies its input, serving as a proxy for the component's impact on the overall computation.

We analyzed the distribution of these ratios across a test set consisting of sum operations for both the 'ALL3' and 'SUM' models. Kernel Density Estimation (KDE) plots were used to visualize the distributions, and we employed several statistical measures to quantify the differences.

Attention Sublayer The attention sublayer showed moderate but statistically significant differences between the 'ALL3' and 'SUM' models:

- Kolmogorov-Smirnov test: statistic = 0.1592, p-value ¡ 0.0001
- Jensen-Shannon divergence: 0.1591
- Wasserstein distance: 0.0696
- Effect size (Cohen's d): 0.1761
- 95% CI for mean difference: (0.0466, 0.0860)

The KDE plot revealed that the 'ALL3' model's attention ratio distribution was more concentrated and peaked higher than the 'SUM' model's distribution. The positive effect size and confidence interval indicate that the 'ALL3' model generally had higher attention ratios.

Feedforward Sublayer The feedforward sublayer exhibited more pronounced differences:

- Kolmogorov-Smirnov test: statistic = 0.2461, p-value ¡ 0.0001
- Jensen-Shannon divergence: 0.1617
- Wasserstein distance: 0.2830
- Effect size (Cohen's d): -0.3379
- 95% CI for mean difference: (-0.3042, -0.2226)

The KDE plot for the feedforward ratios showed a clear shift between the two distributions. The 'SUM' model's distribution was shifted towards higher values, as confirmed by the negative effect size and confidence interval.

Interpretation These results reveal distinct operational patterns between models trained on 'ALL3' operations versus those trained solely on 'SUM':

1. In the attention sublayer, the 'ALL3' model shows slightly higher ratio values, **suggesting that attention mechanisms play a more pronounced role when the model is trained on diverse operations.** This could indicate that the attention sublayer is capturing more complex patterns or relationships necessary for handling multiple operations.
2. Conversely, in the feedforward sublayer, the 'SUM' model demonstrates significantly higher ratio values. This **suggests that when trained on 'Sum' alone, the model relies more heavily on the feedforward network for computation.** This could imply that the 'SUM' operation is being implemented more directly through feedforward transformations.
3. The larger effect size in the feedforward layer (-0.3379) compared to the attention layer (0.1761) indicates that the difference in behavior is more pronounced in the feedforward component.

These observations suggest a trade-off in how the network allocates its computational resources. The 'ALL3' model appears to leverage its attention mechanism more, potentially to handle the diversity of operations it was trained on. In contrast, the 'SUM' model seems to channel more of its computation through the feedforward network, possibly developing a more specialized but less flexible approach to solving the sum operation.

This analysis provides evidence that the internal dynamics of transformer models adapt significantly based on the diversity of tasks they are trained on, even when evaluated on the same type of operation ('SUM'). It highlights the importance of considering task diversity in understanding and optimizing neural network architectures.

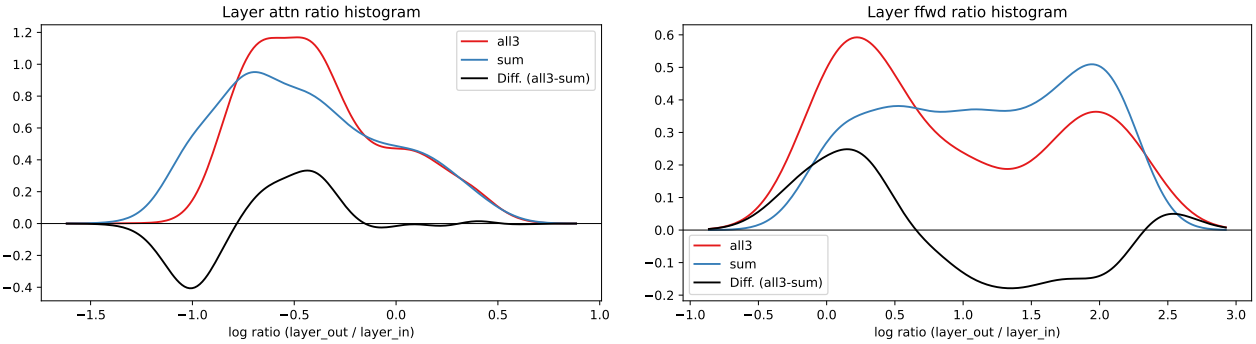


Figure 39. **Comparison of layer output/input ratio distributions** for models trained on all three operations (all3 - MAX+MED+SUM) versus sum operation alone (SUM). **Left:** Attention layer ratio histogram. **Right:** Feedforward layer ratio histogram. The x-axis represents the log ratio of layer output norm to input norm, while the y-axis shows the density. The black line represents the difference between the all3 and sum distributions (all3 - sum). These plots illustrate distinct operational patterns between the two models, with the attention layer showing increased activity in the all3 model and the feedforward layer demonstrating higher ratios in the sum model.

H. Ablation Studies

H.1. Number of Attention Heads

Our main experiments use a single attention head for simplicity and interpretability. To verify that our findings are not an artifact of this choice, we conducted a systematic sweep comparing 1 vs. 4 attention heads across the same range of embedding dimensions and task combinations. We found that the qualitative pattern of results—including the relative ordering of task difficulty (ADD hardest, MAX/MIN easiest) and the effectiveness of specific task pairings (easy+hard helps, hard+hard does not)—remains unchanged across head configurations. The transition points shift slightly but the relative benefits of joint training (2–7$\times$ reduction in model size for successful pairings) persist. This suggests that our findings about task compatibility reflect properties of the tasks themselves rather than artifacts of the single-head architecture.

H.2. Deep vs. Recurrent Architecture

Our primary architecture uses a recurrent transformer: a single transformer block applied iteratively, feeding its output back as input for the next iteration. We compared this to standard deep transformers with 2–4 stacked layers and found

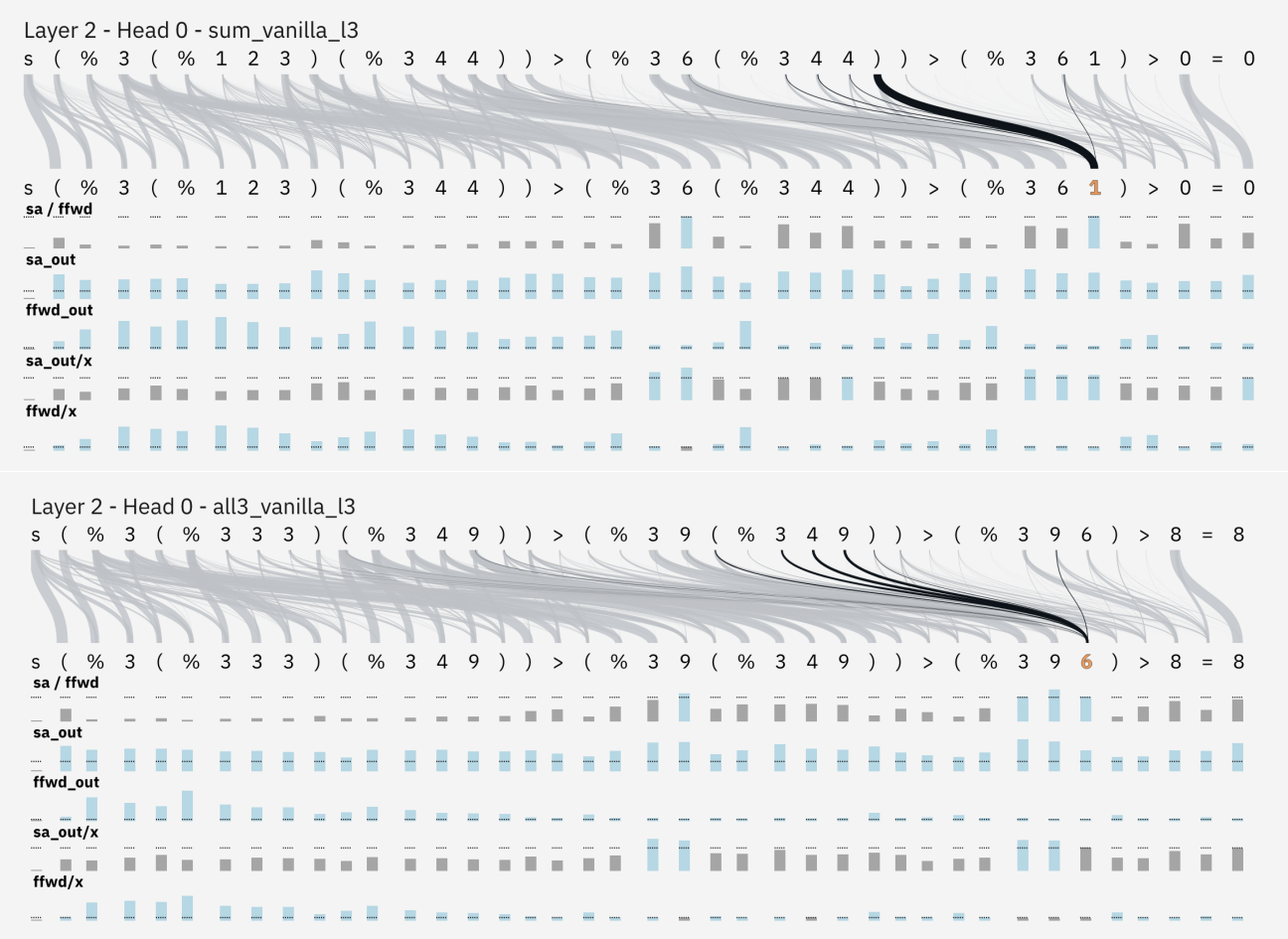


Figure 40. **Attention patterns and layer dynamics in SUM vs MAX+MED+SUM (all3) models.** Each panel shows (CoT) solution to a SUM modulo 10 problem, where '¿' indicates solution steps. The first row shows the input sequence, with curved lines representing attention weights from Layer 2 in a 3-layer network. Black lines highlight attention patterns for a specific digit (shown in orange). Below are shown various layer metrics including the ratio of self-attention to feedforward norms (`sa/ffwd`), self-attention output norms (`sa_out`), feedforward output norms (`ffwd_out`), and ratios of layer outputs to inputs (`sa_out/x`, `ffwd/x`). **Top:** Model trained only on SUM operations shows attention primarily focused on parentheses and structural elements. **Bottom:** Model trained on MAX+MED+SUM (all3) shows attention strongly connecting to digits being combined in each CoT step, suggesting direct involvement in numerical computation. These distinct patterns suggest fundamentally different algorithms learned by each model.

qualitatively similar results for the task difficulty hierarchy and joint training benefits (see Section F). The recurrent architecture was chosen for two reasons: (1) it converges faster during training, and (2) it produces more structured, interpretable embeddings that facilitate our PCA analysis. The key finding—that joint training with compatible tasks reduces model size requirements—holds for both architectures.

I. Additional Scaling Results

Figure 44 shows the full scaling results across all task combinations tested, complementing the selected comparisons in the main text.

J. Additional Curriculum Experiments

Figure 45 shows the full curriculum training results including loss, accuracy, and mix ratio curves comparing baseline ADD training with the ADD+PROD $\rightarrow$ ADD curriculum.

In addition, we tested transfer from MAX+MED to ADD.

MAX+MED $\rightarrow$ ADD transfer. We trained a small model ($n_{\text{embed}} = 48$, below the pure ADD transition) on MAX+MED until proficiency, then gradually transitioned to pure ADD (never showing mixed MAX+MED+ADD expressions). Key findings:

- The model learns ADD immediately upon introduction, achieving high accuracy.
- The model forgets MAX and MED as they are phased out, but retains the structured embedding.
- Models as small as $n_{\text{embed}} = 24$ can learn ADD this way—7$\times$ smaller than the pure ADD transition.

This confirms that the curriculum learning effect is robust across different task combinations: structured representations learned from easy tasks enable efficient learning of hard tasks.

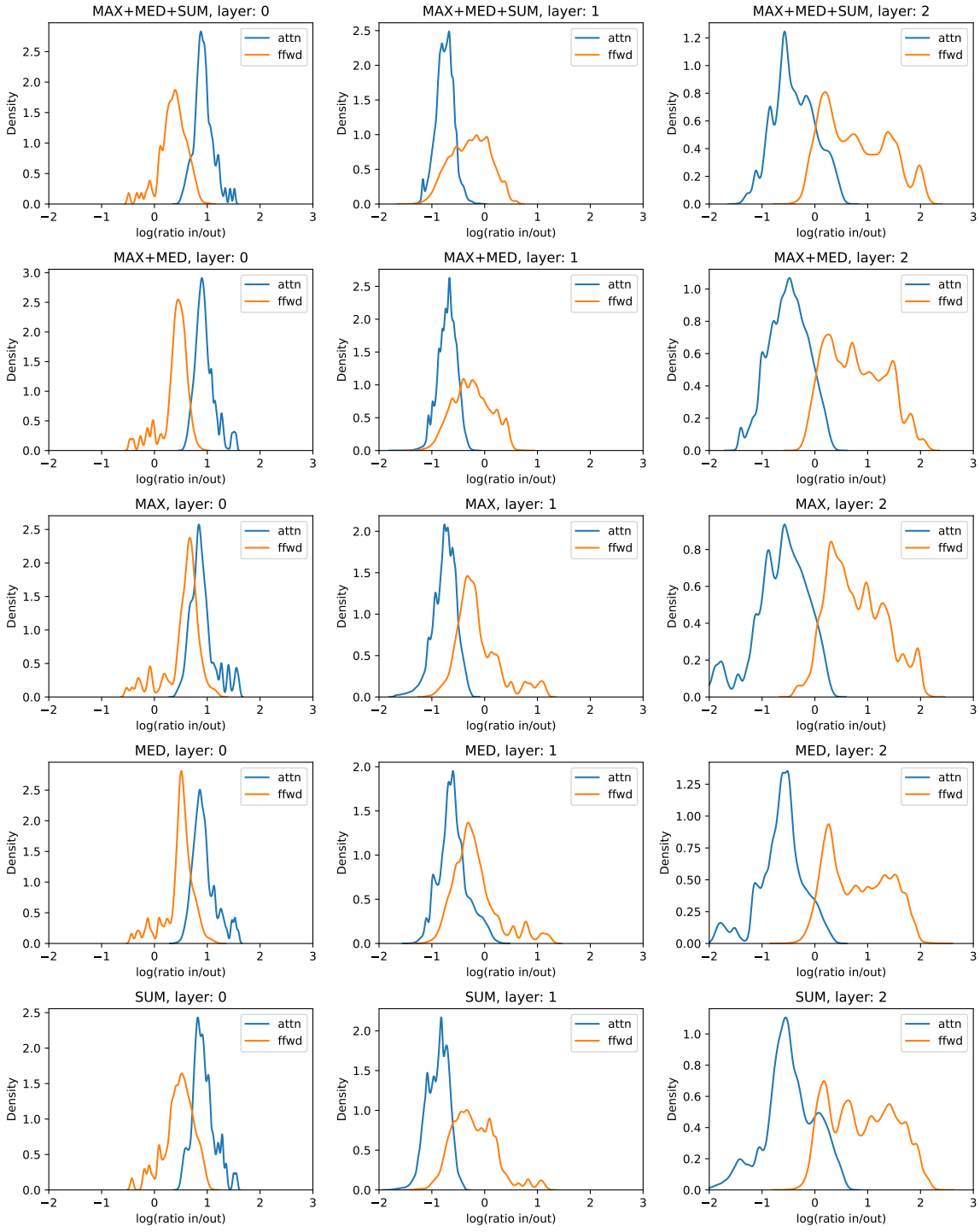


Figure 41. **Attention layer and Feedforward layer ratio histogram.** Each row shows the attention and feedforward layer ratio histogram for models trained on MAX, MED, SUM, MAX+MED,and MAX+MED+SUM. Each column shows the ratio histogram in different attention blocks. The model had 128 embedding and 3 layers and all models reaches 99% accuracy.

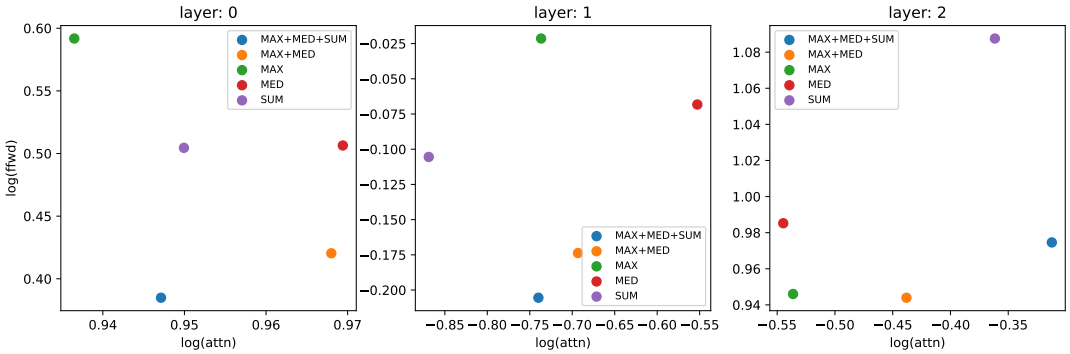


Figure 42. **The mean attention layer ratio vs the feedforward layer ratio.** Each plots show the means in different layers.

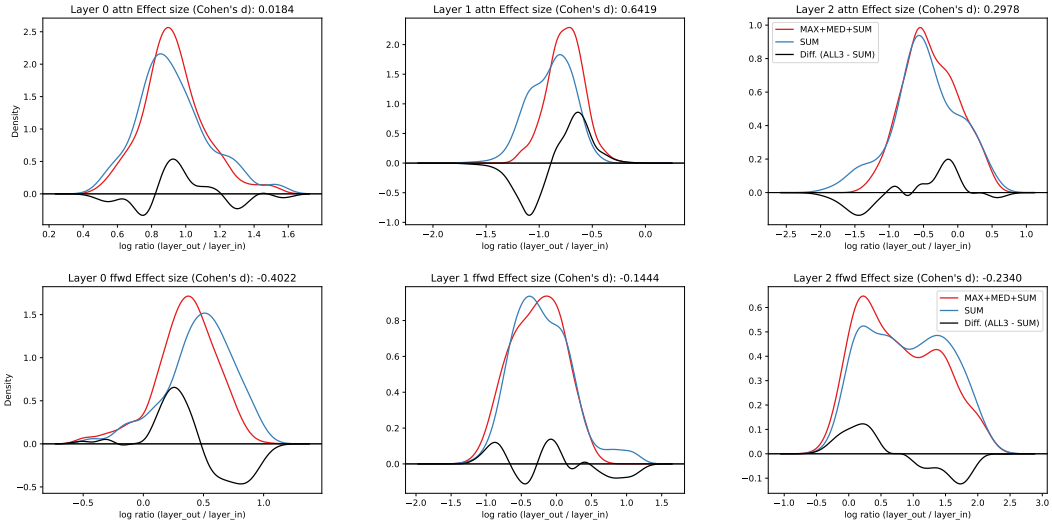


Figure 43. **Comparison of layer output/input ratio distributions** for models trained on all three operations (MAX+MED+SUM) versus sum operation alone (SUM). The title of the figures contain the Effective size showing the difference between the model.

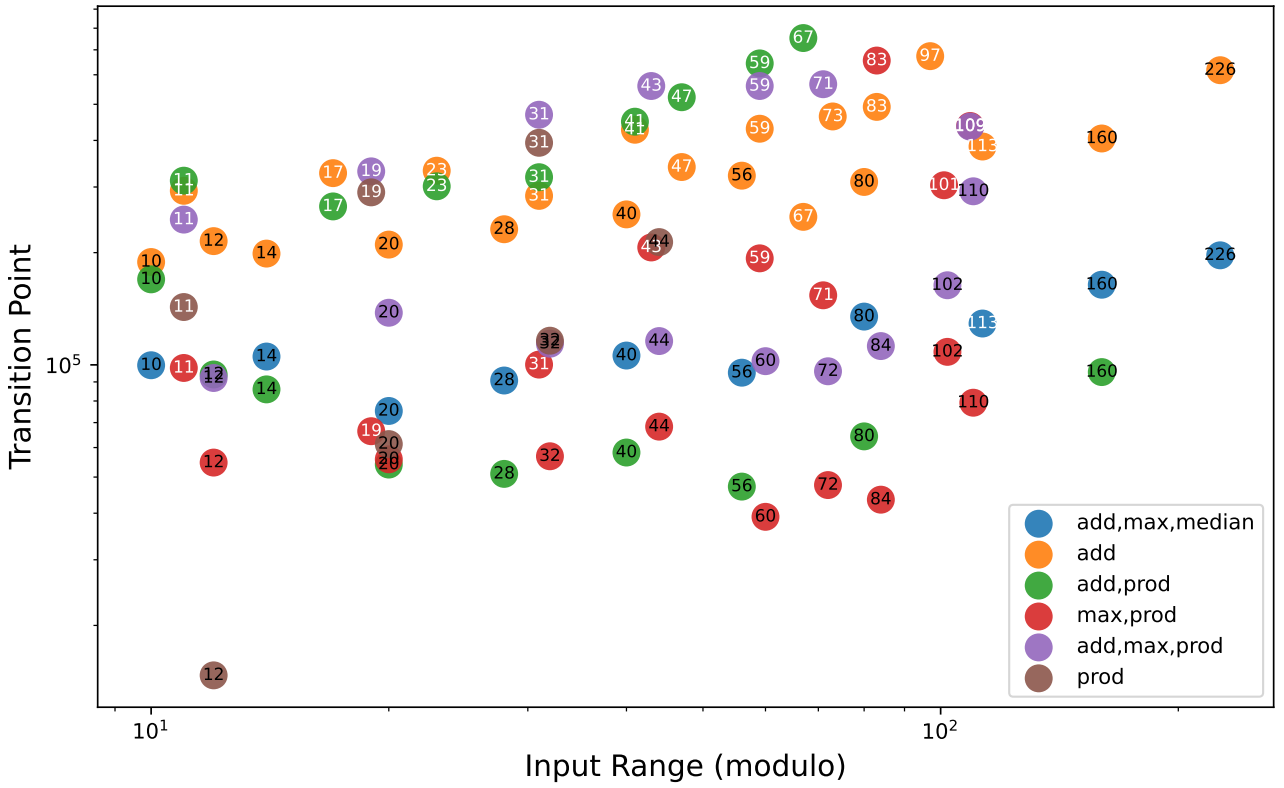


Figure 44. **Full scaling results across moduli.** Transition points (minimum model size to learn) as a function of modulo n for various task combinations. Prime moduli are shown in white text. Key observations: (1) PROD-prime is consistently hard; (2) ADD+PROD helps ADD for non-prime moduli; (3) MAX+PROD helps PROD even for primes; (4) ADD+MAX+PROD at primes can be harder than individual tasks.

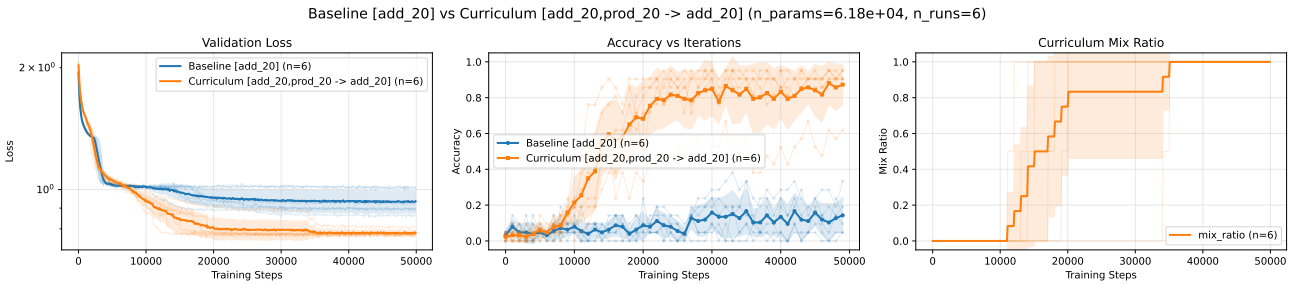


Figure 45. **Full curriculum comparison: ADD+PROD → ADD.** Left: Validation loss over training. Middle: Accuracy over training—baseline ADD (blue) stays at ∼15%, curriculum (orange) reaches near-perfect accuracy. Right: Curriculum mix ratio showing transition from ADD+PROD to pure ADD.

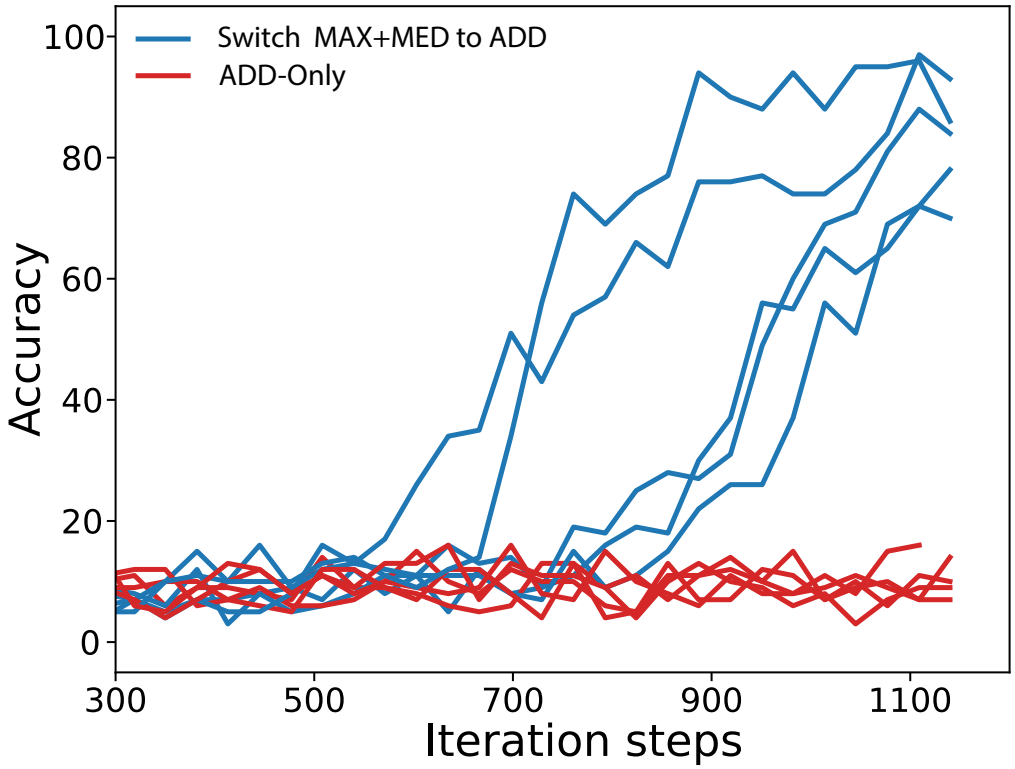


Figure 46. **Transfer from MAX+MED to ADD.** A model too small to learn pure ADD ($n_{\text{embed}} = 48$) successfully learns ADD after pretraining on MAX+MED. Blue: curriculum (MAX+MED $\rightarrow$ ADD) reaches high accuracy. Red: baseline (pure ADD) fails to learn.

K. Complexity: Number of operands and nesting level.

In previous studies, task complexity has been characterized through various measures, including the number of bits required to memorize the task, which corresponds to the length of the expression (Dave et al., 2024), the number of operands and nesting depth (Petruzzellis et al., 2024), and the structure of the computational graph (Dziri et al., 2023), which captures the number of nestings. Here, we examine how the number of operands and nesting levels affect the learning ability of small language models. Focusing on the `all3` task, which combines `max`, `med`, and `sum` operations, we manipulate complexity by varying the number of operands (`arg = {3, 4, 5}`) and nesting depth (`depth = {3, 4, 5}`). The length of equations, measured by the number of characters, depends on both the number of operands and the nesting level. We find that nesting has a greater impact: increasing the nesting level from 3 to 4 results in longer equations than increasing the number of operands from 3 to 5 at a fixed nesting level. We train the GPT model on the `all3` dataset with all combinations of `arg` (number of operands) and `depth` (nesting levels), finding that the model's performance correlates with the sum of operands and nesting levels (`arg + depth`). Notably, transition points tend to group together for configurations with the same sum (Fig. 47, 48b). While Fig. 48a demonstrates that the model requires more parameters to solve longer equations, it also indicates that `arg + depth` serves as a reliable predictor of the transition point.

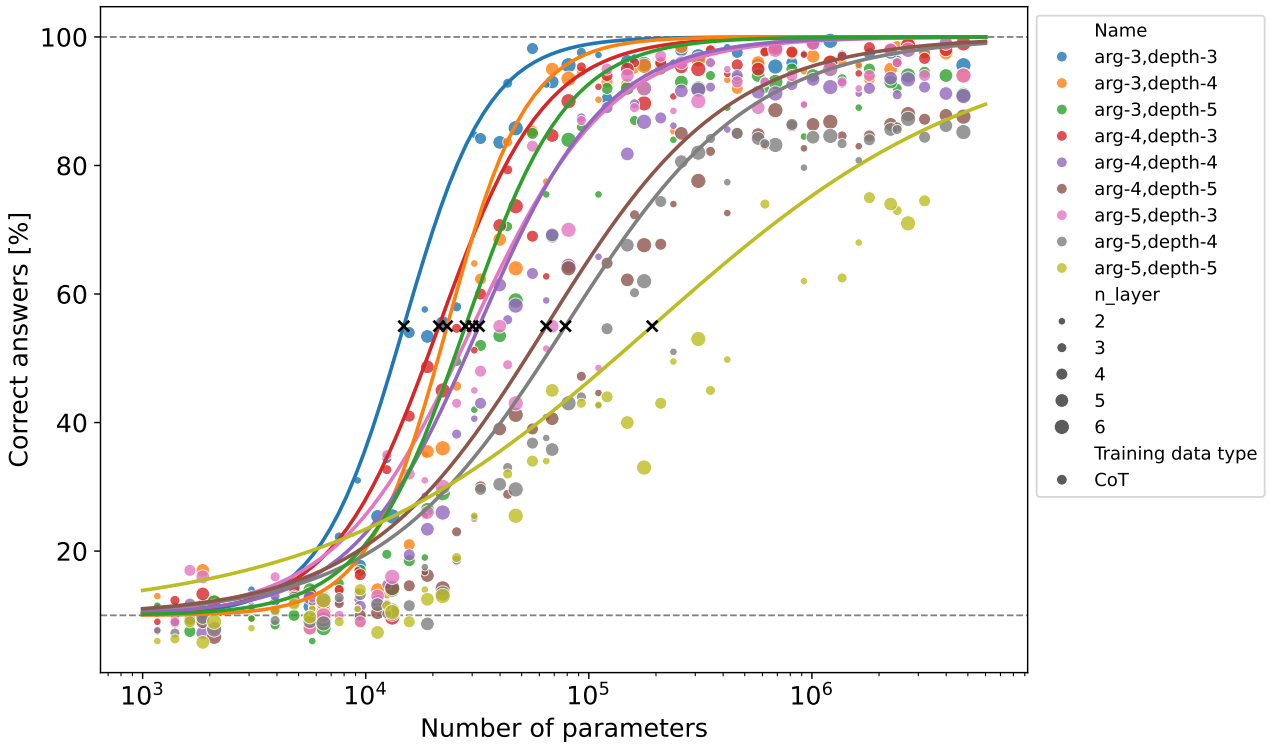
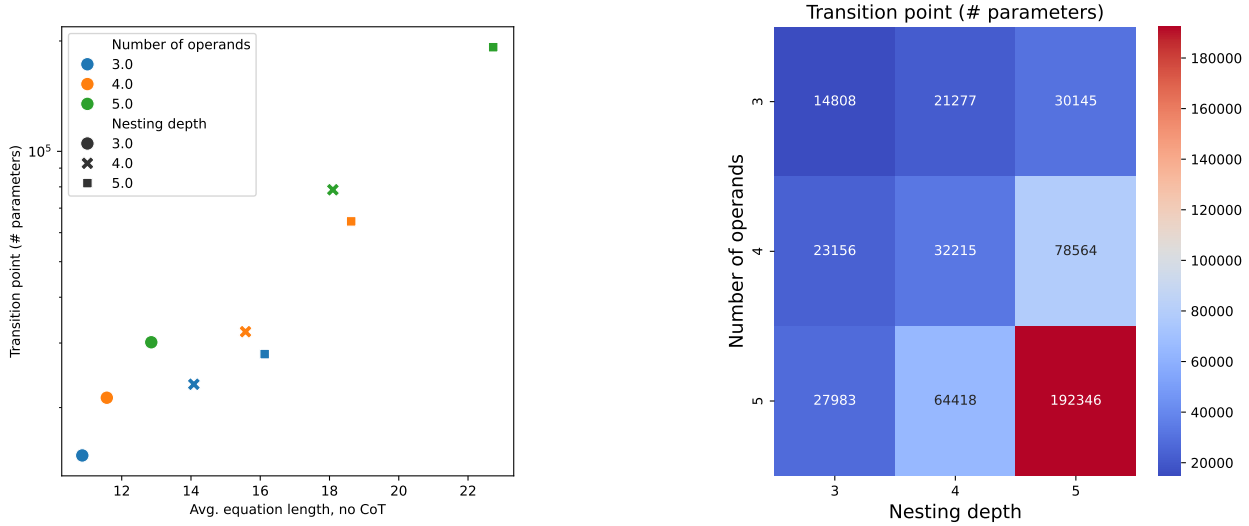


Figure 47. **Learning MAX+MED+SUM operations with varying numbers of operands and nesting levels.** The model requires more parameters as the number of operands and nesting levels increases. Higher nesting levels particularly demand larger model sizes to learn the task. We present the average of five simulations for each configuration and fit a sigmoid function, with the cross marking the middle value (transition point). The transition points reveal an interesting pattern: the sum of the number of operands and nesting levels groups together. For example, the transition points for arg-3,depth-4 (orange) and arg-4,depth-3 (red) are close to each other, as are those for arg-4,depth-5 (brown), and arg-5,depth-4 (grey). Early stopping criteria were applied during training in all simulations.



(a) **Average equation length vs. transition point.** *(b)* **Transition points.**

Figure 48. **Transition point vs. Equation Lenght and Nesting Depth.** (a) Transition point in function of the average equation length. (b) Heat plot of transition point in function number of operands and nesting depth.

L. Clustering in the embedding space

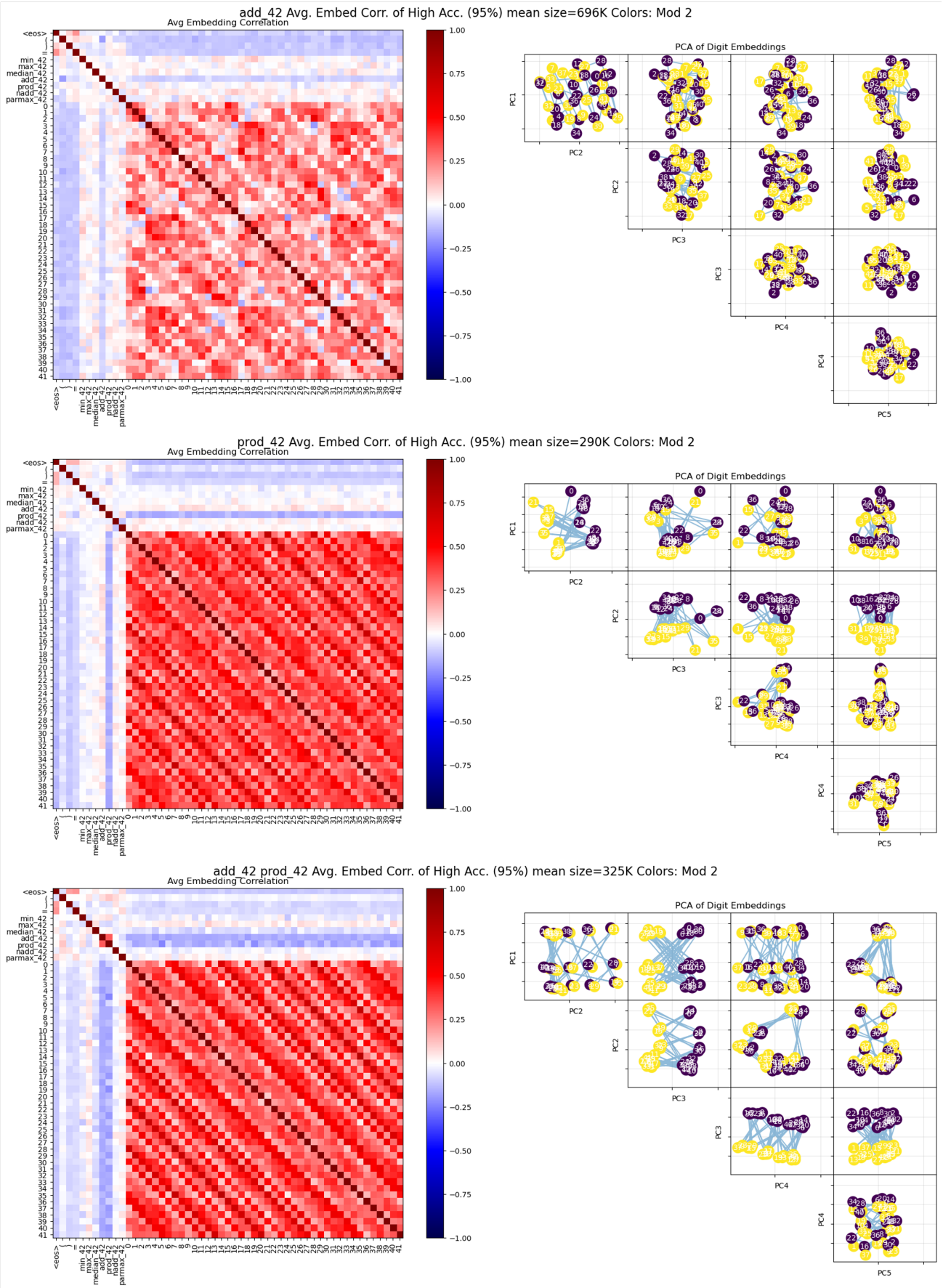


Figure 49. **ADD, PROD and ADD+PRD in MOD 42 PC embeddings**

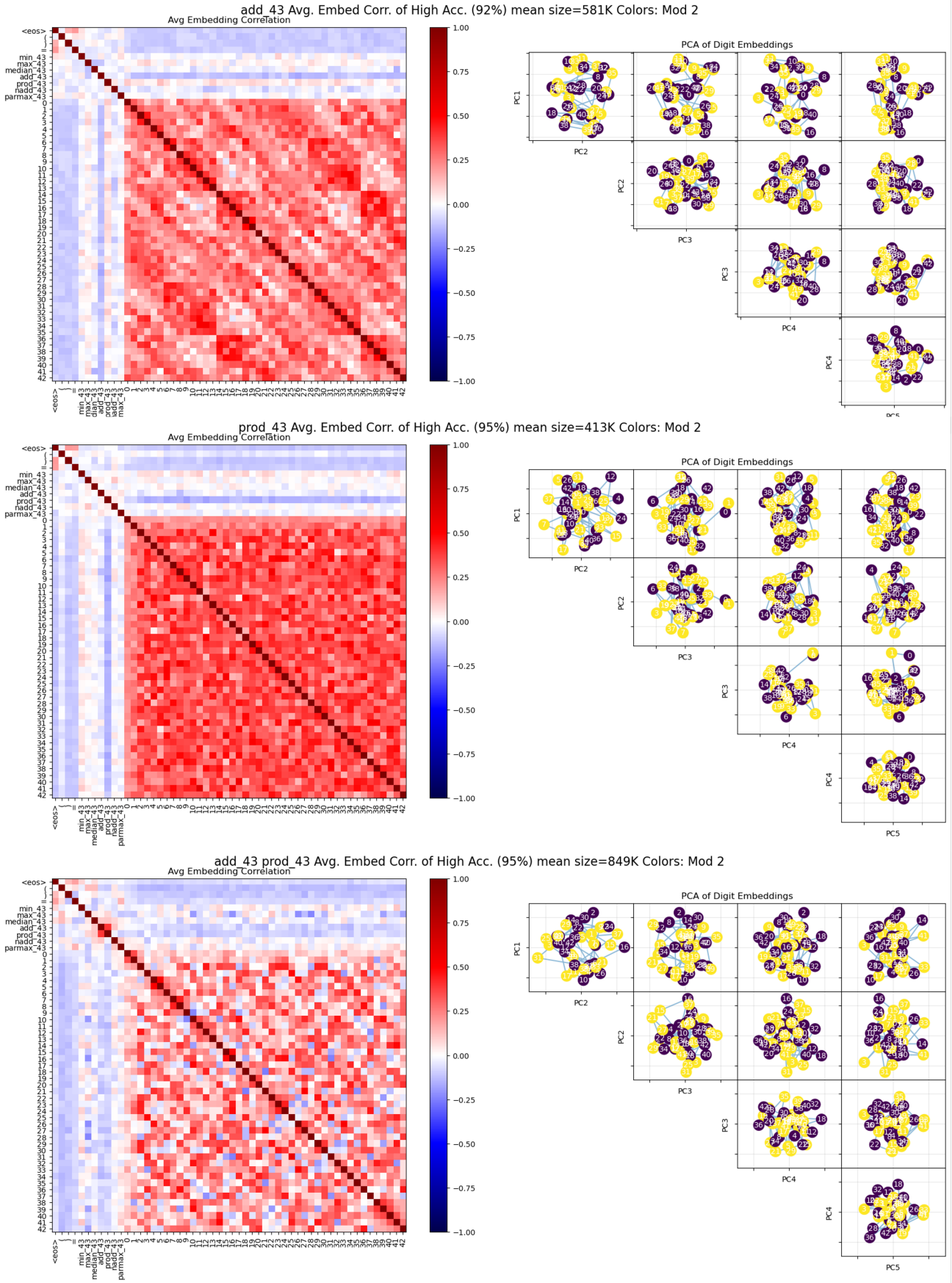


Figure 50. **ADD, PROD and ADD+PRD in MOD 43 PC embeddings**

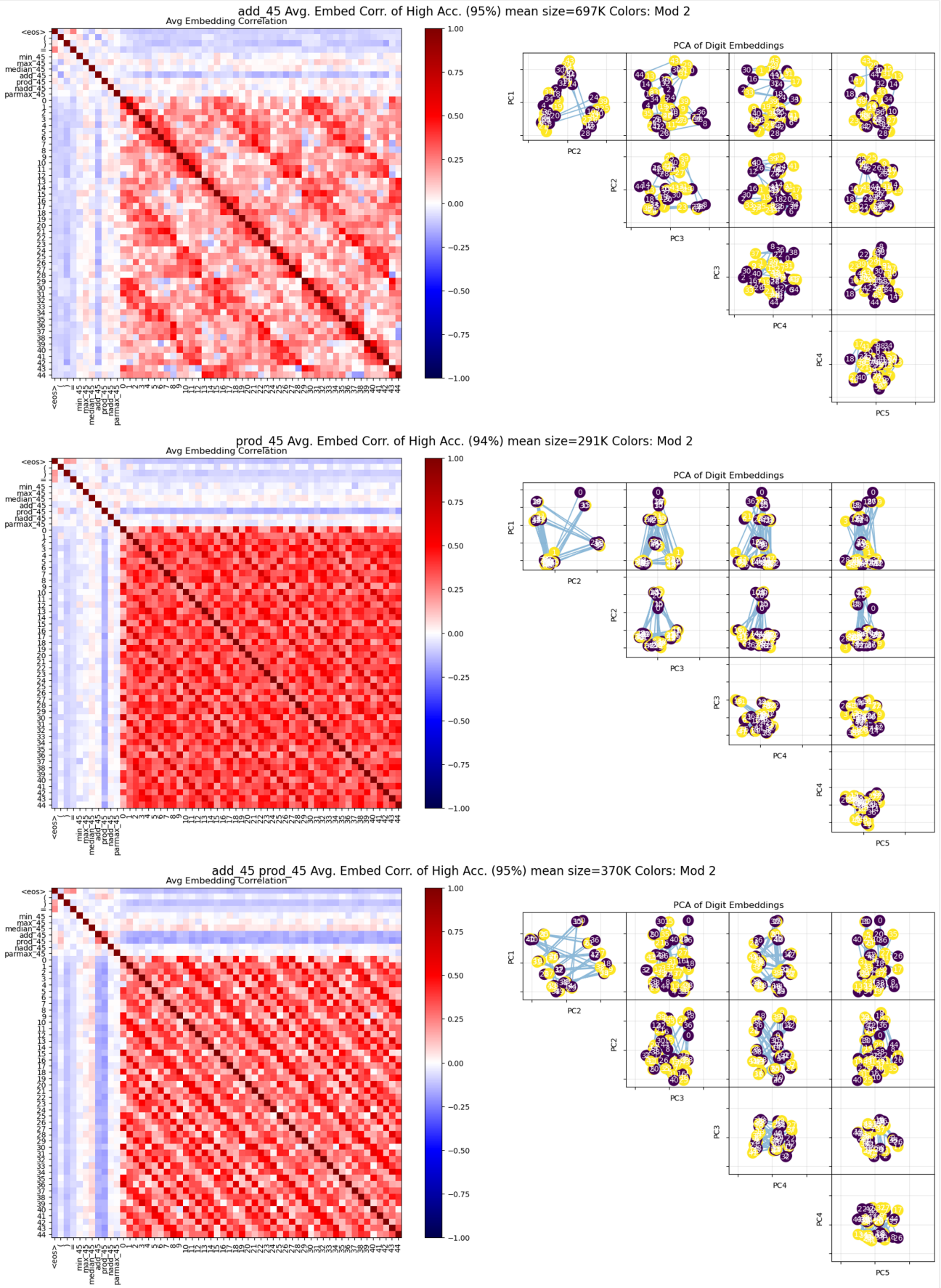


Figure 51. **ADD, PROD and ADD+PRD in MOD 45 PC embeddings**

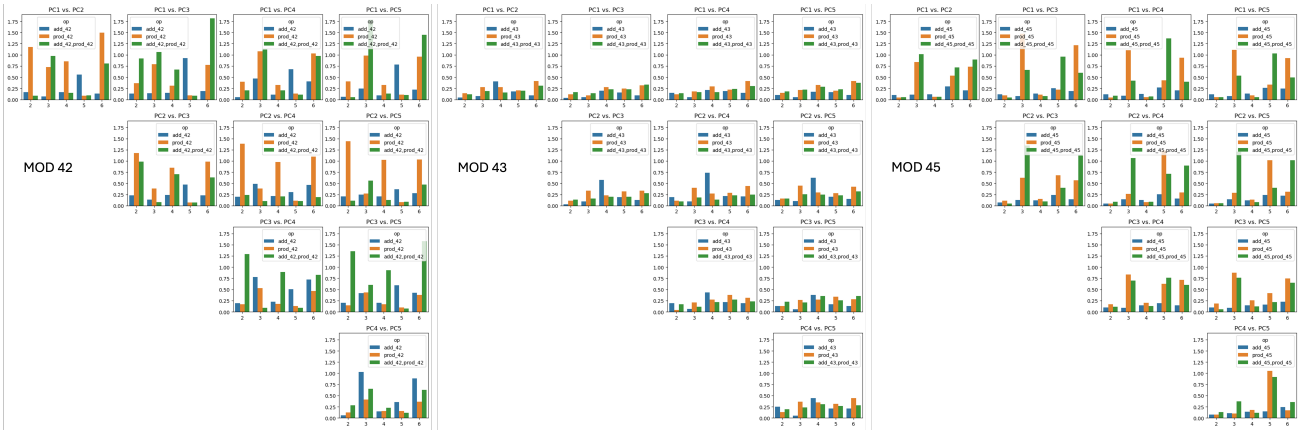


Figure 52. **Clustering in the PC space.** We measure how well modulo-2, 3, 4, 5, and 6 divisor clusters separate in the PC space for the ADD, PROD, and ADD+PROD models on MOD 42, 43, and 45 (for runs with accuracy $> 80\%$). Cluster quality is computed from the mean position and standard deviation of each modulo group, and the pairwise distances between them. For MOD 42, strong clustering appears for mod-2, 3, 6, and sometimes 4—precisely the divisors of 42. Strong PROD signals almost always produce strong ADD+PROD signals, whereas strong ADD signals without PROD support do not transfer. For MOD 43, no model shows strong clustering; both ADD and PROD struggle and do not help each other. For MOD 45, strong clustering occurs for mod-3 and mod-5, and occasionally mod-6 (via divisor 3). We also observe that the mixed model only exhibits strong clustering when PROD does as well, indicating that the PROD embedding drives the shared structure and helps ADD.

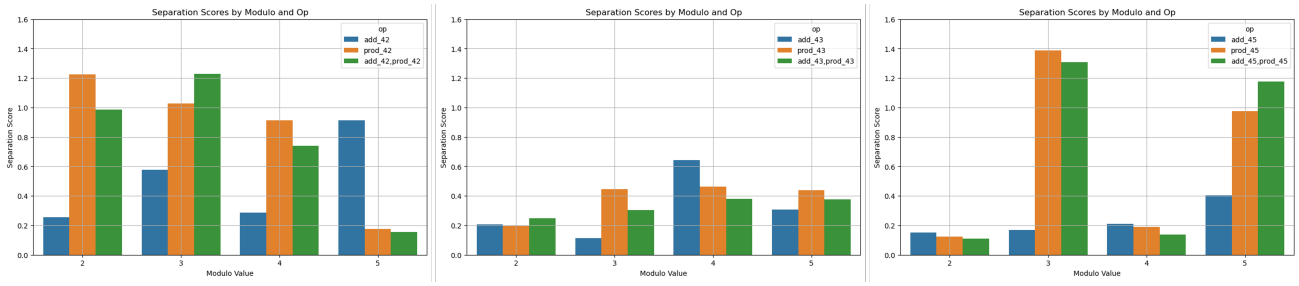


Figure 53. **Clustering in the PC space by averaging over the first 5 PCs.** We measure how well modulo-2, 3, 4, 5, and 6 divisor clusters separate in the PC space for the ADD, PROD, and ADD+PROD models on MOD 42, 43, and 45 (for runs with accuracy $> 80\%$). Cluster quality is computed from the mean position and standard deviation of each modulo group, and the pairwise distances between them, averaged over the first five PCs. For MOD 42, strong clustering appears for mod-2, 3, and 4 (could be an artifact of 2) — precisely the divisors of 42. For MOD 43, we find no strong clustering compared to MOD 42 and MOD 45. For MOD 45, strong clustering occurs for mod-3 and mod-5, as expected from the divisors of 45. We also observe that the mixed model exhibits strong clustering only when PROD does as well, indicating that the PROD embedding drives the shared structure and supports ADD.